Financial Fraud Detection Using Graph Machine Learning

A PROJECT REPORT

Submitted By

AARJAV ORAVAKANDI

TVE25MCA-2002

To

The APJ Abdul Kalam Technological University
in partial fulfilment of the award of the degree
of
Master of Computer Applications



Department of Computer Applications
College of Engineering Trivandrum
Trivandrum
November 2026

Declaration

I, the undersigned, hereby declare that the project report titled “Financial Fraud Detection Using Graph Machine Learning” submitted for partial fulfilment of the requirements for the award of the degree of Master of Computer Applications of the APJ Abdul Kalam Technological University, Kerala, is a bona fide work carried out by me under the supervision of Dr. Haripriya A P, Head of the Department. This submission represents my own ideas in my own words; where the ideas or words of others have been included, I have adequately and accurately cited and referenced the original sources. I also declare that I have adhered to the ethics of academic honesty and integrity as directed in the ethics policy of the college and have not misrepresented or fabricated any data or idea or fact or source in my submission. I understand that any violation of the above will be a cause for disciplinary action by the Institute and/or University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not previously formed the basis for the award of any degree, diploma, or similar title.

Place : Trivandrum
Date : AARJAV ORAVAKANDI

COLLEGE OF ENGINEERING TRIVANDRUM

THIRUVANANTHAPURAM

695016



CERTIFICATE

This is to certify that the report entitled **Financial Fraud Detection Using Graph Machine Learning** submitted by **Aarjav Oravakandi** (Reg No. TVE25MCA-2002) to the APJ Abdul Kalam Technological University in partial fulfilment of the requirements for the award of the Degree of Master of Computer Applications is a Bonafide record of the project work carried out by him under my guidance and supervision. This report in any form has not been submitted to any University or Institute for any purpose.

Internal Supervisor External Examiner

Head of the Department

Acknowledgment

First and foremost I thank GOD almighty and my parents for the success of this project. I owe sincere gratitude and heartfelt thanks to everyone who shared their time and knowledge during the course of this work.

I am thankful to **Dr. Joseph Zacharias**, Principal, College of Engineering Trivandrum, for providing the facilities and the academic environment in which this project was carried out.

I am deeply grateful to **Dr. Haripriya A P**, Head of the Department of Computer Applications, who also supervised this project, for guidance on the scope and direction of the work, for a detailed review of the design and the results, and for constant encouragement throughout.

I thank the faculty members of the Department of Computer Applications and the staff of the College of Engineering Trivandrum for their instruction and support throughout my course of study.

I thank my classmates, who tested the deployed application and reported several defects that are described in Chapter 6, and my family for their constant support.

Aarjav Oravakandi

Abstract

Financial fraud is rarely the act of a single account. Funds are divided into small transfers and routed through intermediate accounts, so that each transaction appears unremarkable when examined on its own. Detection methods that score transactions or accounts in isolation are, by construction, unable to use the relationships between them.

This project implements a fraud detection system that represents the transaction history as a **directed, weighted graph** in which customer accounts are nodes and transfers are edges, and layers several detection techniques over that representation. A **rule engine** applies six deterministic checks derived from common anti-money-laundering typologies and records, for every flagged transaction, which rule fired. A **graph analysis module** built on NetworkX computes PageRank [1], degree centrality, Louvain community membership [2] and cycle membership for each account. A **feature engineering** stage combines eleven transaction-level aggregates with six graph-derived measures into a seventeen-feature vector per account. Three classical models are trained on that matrix — Random Forest [3] and XGBoost [4] as supervised classifiers, and Isolation Forest [5] as an unsupervised anomaly detector — and two graph neural networks, a Graph Convolutional Network [6] and GraphSAGE [7], are trained on the account graph itself. Connected groups of suspicious accounts are assembled into **fraud rings** with member roles inferred from money flow, and each ring or alert can be escalated into a tracked **investigation case**. The analyst interface is a React single-page application served by a FastAPI backend over a SQLite database accessed through the SQLAlchemy ORM.

Evaluation uses a synthetic dataset of 5,000 accounts and 80,000 transactions with planted fraud structures, which provides complete ground truth. On that dataset the rule engine recovered 96.21% of fraudulent transactions at 99.88% precision; XGBoost reached an F1 score of 0.825 and GraphSAGE 0.839 on held-out accounts; and 52 fraud rings were assembled with 96.8% of their members genuinely fraudulent. The graph neural networks were additionally evaluated on the public Elliptic Bitcoin dataset [8], where GraphSAGE reached an F1 score of 0.549 under the dataset's temporal split.

Because the primary dataset is synthetic and its fraud structures were planted by its own generator, these figures measure internal consistency rather than performance on live banking data; the system is a working prototype and is not presented as production-ready. The report documents the requirement analysis, design, implementation, testing and evaluation of the system, and states its limitations explicitly in Chapter 7.

Keywords: Graph Machine Learning, Financial Fraud Detection, NetworkX, PageRank, Community Detection, Fraud Rings, XGBoost, Feature Engineering, FastAPI, React

Contents

Abstract

List of Figures

List of Tables

1 Introduction **1**
- 1.1 Project Objectives . . . 2
- 1.2 Literature Survey . . . 2
  - 1.2.1 Rule-Based and Transaction-Level Detection . . . 2
  - 1.2.2 Relationship-Aware and Graph-Based Detection . . . 3
  - 1.2.3 Graph Neural Networks . . . 3
  - 1.2.4 Datasets and Benchmarking . . . 3
  - 1.2.5 Position of This Work . . . 4
- 1.3 Report Organisation . . . 4

2 Problem Definition and Motivation **5**
- 2.1 Existing System . . . 5
  - 2.1.1 Limitations of Existing System . . . 6
- 2.2 Proposed System . . . 6
  - 2.2.1 Advantages of Proposed System . . . 7

3 Requirement Analysis **9**
- 3.1 Purpose . . . 9
- 3.2 Overall Description . . . 9
  - 3.2.1 Hardware Requirements . . . 10
  - 3.2.2 Software Requirements . . . 10
- 3.3 Functional Requirements . . . 11
  - 3.3.1 Python and FastAPI . . . 12
  - 3.3.2 NetworkX and Graph Algorithms . . . 12
  - 3.3.3 PyTorch Geometric and Graph Neural Networks . . . 12

3.3.4 SQLite and SQLAlchemy . . . 13
3.3.5 React and Material UI . . . 13
3.3.6 Fraud Rule Specification . . . 13
3.4 Non-Functional Requirements . . . 15
3.4.1 Performance Requirements . . . 15
3.4.2 Security Requirements . . . 15
3.4.3 Reliability and Scalability . . . 15

4 Design and Implementation 17
4.1 Overall Design . . . 17
4.2 System Design . . . 17
4.2.1 Database Schema . . . 17
4.2.2 Backend Architecture . . . 18
4.2.3 Detection Pipeline Orchestration . . . 20
4.2.4 Analyst Review and Response . . . 20
4.2.5 Demonstration Scenario . . . 20
4.2.6 Frontend Framework (React) . . . 21
4.3 Use Case Diagram . . . 21
4.4 Context and Data Flow Diagrams . . . 22
4.5 Detection Pipeline Flow . . . 22
4.6 Database Design . . . 24
4.6.1 Entity Relationship Diagram . . . 24
4.6.2 Table Definitions . . . 27
4.7 Security Design . . . 29
4.7.1 Authentication and Authorisation . . . 29
4.7.2 Configuration and Transport . . . 30
4.7.3 Audit Trail and Account-Holder Notification . . . 30
4.7.4 Controls Not Implemented . . . 31
4.8 Screenshots of User Interface . . . 31

5 Coding 36
5.1 Python . . . 36
5.2 FastAPI . . . 37
5.3 React . . . 37
5.4 Sample Coding . . . 37
5.4.1 rules.py — Rule Engine . . . 38
5.4.2 graph_analysis.py — Graph Construction and Metrics . . . 38
5.4.3 baseline.py — Model Training and Scoring . . . 40
5.4.4 gnn.py — Graph Neural Network Models . . . 41

6 Testing and Validation **43**
6.1 Testing Methods . . . 43
6.1.1 Unit Testing . . . 43
6.1.2 Integration Testing . . . 44
6.1.3 Alpha Testing . . . 44
6.1.4 Beta Testing . . . 45
6.2 Test Cases . . . 46
6.3 Test Execution Results . . . 48

7 Results and Discussion **50**
7.1 Experimental Setup . . . 50
7.1.1 Dataset . . . 50
7.1.2 Preprocessing, Splits and Hyperparameters . . . 52
7.2 Results . . . 54
7.2.1 Rule Engine . . . 54
7.2.2 Model Performance . . . 54
7.2.3 Feature Importance . . . 57
7.2.4 Fraud Ring Detection . . . 58
7.2.5 Elliptic Bitcoin Benchmark . . . 58
7.2.6 Runtime . . . 58
7.2.7 Reproducibility . . . 59
7.3 Advantages and Limitations . . . 62
7.3.1 Advantages . . . 62
7.3.2 Limitations . . . 64

8 Conclusion and Future Scope **66**

Bibliography **68**

List of Figures

4.1 High-Level System Architecture 19
4.2 Use Case Diagram 23
4.3 Context-Level Data Flow Diagram (Level 0) 23
4.4 Level-1 Data Flow Diagram 24
4.5 Detection Pipeline Data Flow Diagram 25
4.6 Entity Relationship Diagram 26
4.7 User Interface — Authentication and Dashboard 32
4.8 User Interface — Accounts and Graph Analysis 33
4.9 Feature Engineering View — Per-Account Feature Matrix and Model Scores 34
4.10 Demonstration Scenario — static screenshot of the interactive money-flow network of Section 4.2.5 34
4.11 Account Investigation — a confirmed fraud account showing the analyst and timestamp that confirmed it, the rule-based risk level alongside the independent model score, the account-holder notification that was sent on confirmation with the recipient address masked, the audit trail of status transitions with the analyst's notes, and the account's suspicious transactions 35

7.1 Model performance on the synthetic account graph. Classical models (left three) and graph neural networks (right two) are evaluated on different partitions of the same data. 55
7.2 ROC and precision–recall curves for the two graph neural networks on the synthetic account graph, computed from the stored per-node test predictions. 56
7.3 Confusion matrices for GraphSAGE at the 0.5 decision threshold. Left: the synthetic account graph. Right: the Elliptic Bitcoin dataset. 56
7.4 Training and validation curves over 100 epochs. Neither model shows a widening train–validation gap, so overfitting is not evident at this depth. . 57
7.5 XGBoost gain importance, eight highest-ranked features. 57

List of Tables

3.1 Minimum Hardware Requirements 10
3.2 Software Requirements 10
3.3 Implemented Fraud Rules 14

4.1 Backend Persistence Schema 18
4.2 Account Feature Vector — the seventeen columns of `FEATURE_COLS` . . . 29

6.1 Unit Test Files and Coverage Areas 44
6.2 Integration Test Files 44
6.3 Test Cases 46
6.4 Test Suite Execution Summary 49

7.1 Dataset population used by each experiment 51
7.2 Model Configuration 53
7.3 Model Performance on the Synthetic Account Graph 55
7.4 Graph Neural Network Results on the Elliptic Bitcoin Dataset 58
7.5 Evaluation Provenance and Reproduction Environment 60
7.6 Evidential Status of the Chapter 7 Results 61
7.7 Result Artefacts 62

Chapter 1

Introduction

The rapid growth of digital payment systems, instant transfers and online banking has been accompanied by an equally rapid evolution in financial fraud. Fraud today is rarely a single suspicious transaction carried out by a lone actor; it is a coordinated effort distributed across many accounts that are deliberately structured to look ordinary when examined one at a time. A launderer rarely moves a large sum through a single transfer. Instead the amount is broken into smaller transactions and routed through a network of intermediate accounts—often called mule accounts—before being consolidated and withdrawn.

Financial Fraud Detection Using Graph Machine Learning is a response to this shift. Rather than treating the transaction history as a flat list of independent events, the system models it as a **network**: customer accounts become **nodes** and transactions between them become **directed, weighted edges**. Once the data is represented this way, the structural fingerprints that coordinated fraud leaves behind—many accounts converging on a single collector, one account dispersing funds to many others, or money travelling in a closed loop—become measurable properties of the graph rather than patterns that a human must notice by eye.

The design of the system prioritises three principles:

1. **Explainability**: Every detection carries a reason, though the reasons are not all of the same kind. A rule hit names the business rule that was violated; a graph metric such as PageRank or degree centrality is itself a readable quantity; a classical model contributes a probability together with the global feature importances of the trained ensemble; and a graph neural network contributes a probability and nothing more, its learned node embeddings being opaque. Section 7.3.1 sets out the distinction in full.
2. **Layering**: No single technique is relied upon. Transparent rules, structural graph analysis and learned models each cover the blind spots of the others, so that a fraudster who evades one layer is still exposed by the next.
3. **Actionability**: Detection is not the end of the process. Scores and rule hits are converted into alerts, alerts are grouped into fraud rings, and rings are carried through to

a tracked investigation case with a documented final decision.

The system is delivered as a full-stack web application. A FastAPI backend hosts the detection pipeline and exposes a REST API; a React single-page application provides the fraud analyst interface; and a SQLite database accessed through the SQLAlchemy ORM persists accounts, transactions, computed graph metrics, alerts and investigation cases.

1.1 Project Objectives

The work set out to achieve the following, each of which corresponds to a capability that is implemented and evaluated in this report:

1. Represent a financial transaction history as a directed, weighted graph of accounts and transfers, rebuilt on demand from relational data.
2. Detect suspicious activity through a set of explicit, auditable fraud rules that state the reason for every flag.
3. Identify structurally suspicious accounts using established graph algorithms — PageRank [1], degree centrality, Louvain community detection [2] and cycle detection.
4. Engineer a per-account feature vector that combines transaction-level aggregates with graph-derived measures.
5. Train and evaluate supervised classifiers, an unsupervised anomaly detector, and two graph neural network architectures on that data.
6. Group connected suspicious accounts into candidate fraud rings and infer each member's role from the direction of money flow.
7. Provide an analyst interface that carries a detection through review, case management and reporting.
8. Measure detection effectiveness with metrics appropriate to a heavily imbalanced problem, on a labelled synthetic dataset and on an independent public benchmark.

1.2 Literature Survey

1.2.1 Rule-Based and Transaction-Level Detection

Statistical and rule-based fraud detection has a long history; Bolton and Hand [12] survey the field and note early on that detection is hampered by extreme class imbalance, by the adversarial nature of the problem, and by the scarcity of labelled data. Threshold rules remain in widespread operational use because they are auditable and need no training data,

but a fixed threshold is also predictable, and an adversary who can infer it can operate beneath it. Supervised models trained on transaction-level or account-level features relax that rigidity, yet when each record is scored from its own attributes the relationships between records are not available to the model.

1.2.2 Relationship-Aware and Graph-Based Detection

Graph-based anomaly detection addresses precisely that gap, and it is an established field rather than a novel one. Pourhabibi et al. [10] review the literature systematically and classify approaches by graph type and detection strategy, concluding that structure-based methods detect organised fraud that feature-based methods miss, and that most published work stops at producing a score. Classical network measures provide the vocabulary: PageRank [1] identifies accounts into which value concentrates, and modularity-based community detection [2] isolates densely connected groups in near-linear time.

1.2.3 Graph Neural Networks

Graph neural networks learn a node representation from its neighbourhood rather than from hand-chosen measurements. Kipf and Welling's Graph Convolutional Network [6] applies a first-order approximation of spectral convolution, aggregating each node with its neighbours through the normalised adjacency matrix; it is transductive and weights all neighbours uniformly. GraphSAGE [7] instead learns aggregator functions over sampled neighbourhoods, which allows it to generalise to nodes unseen during training. Cheng et al. [11] survey the application of these architectures to financial fraud and identify class imbalance and deliberate camouflage as the dominant open problems. Dou et al. [9] address the second directly: their CARE-GNN model selects which neighbours to aggregate, because a fraudster who transacts with many legitimate counterparties can otherwise dilute their own signal under uniform aggregation.

1.2.4 Datasets and Benchmarking

Labelled financial transaction graphs are rarely public. The Elliptic Bitcoin dataset [8] is the most widely used open benchmark: roughly 203,000 transaction nodes with anonymised features and a temporal split. Its authors found that a Random Forest on engineered features was a strong baseline and that graph convolution helped chiefly where local features were weak. Because its features are anonymised and its nodes are transactions rather than accounts, results on it are not directly comparable with results on an account-level graph — a limitation this report observes when reporting its own figures in Chapter 7.

1.2.5 Position of This Work

This project does not introduce a new detection algorithm, and it does not claim to originate graph-based fraud detection. Its contribution is one of integration: it combines a transparent rule layer, classical graph analytics, supervised and unsupervised models, and two graph neural network architectures over a single shared graph, and — addressing the gap Pourhabibi et al. [10] identify — carries the output through ring assembly, analyst confirmation with an audit trail, case management and reporting, so that a detection ends in a recorded decision rather than in a score.

1.3 Report Organisation

The remaining chapters are organised as follows. Chapter 2 defines the problem and motivates the proposed approach. Chapter 3 presents the requirement analysis. Chapter 4 covers the design and implementation. Chapter 5 discusses the coding choices and presents key code snippets. Chapter 6 describes the testing and validation strategy and its results. Chapter 7 presents experimental results, advantages and limitations. Chapter 8 concludes the report and outlines future enhancements.

Chapter 2

Problem Definition and Motivation

This chapter examines the shortcomings of current fraud detection practice that motivated the development of this system. It defines the problem of transaction-level, relationship-blind fraud detection and presents the proposed graph-based solution along with its key design advantages.

2.1 Existing System

Operational fraud monitoring commonly combines three approaches. The characterisation below describes the conventional, transaction-focused configuration of each; it is not a claim that no deployed system performs relationship analysis. Graph-based detection is an established research area and is used in practice [10], and the contribution of this project lies in integration rather than in originating the approach.

Manual review places flagged transactions in front of a human analyst. It is accurate and accountable, but it does not scale: an institution processing millions of transactions cannot staff enough analysts to inspect a meaningful fraction of them, and review queues grow faster than they can be cleared.

Threshold-based rule engines encode fixed limits—for example, flagging any transfer above a set amount. They are transparent and easy to audit, which is why they remain universal, but they are also entirely predictable. A fraudster who learns where a limit sits can stay just beneath it indefinitely.

Transaction-level machine learning models score each transaction or each account from its own history: amounts, frequencies, timing. These models adapt to patterns that no analyst thought to write a rule for, but where the feature set is limited to a record's own attributes the relationships between records cannot inform the score. Two accounts with identical transaction statistics then receive identical treatment, even if one sits at the centre of a fraud ring and the other does not. Class imbalance compounds the difficulty, as Bolton and Hand [12] observe.

2.1.1 Limitations of Existing System

1. **Relationship Blindness**: Where records are evaluated one at a time, coordination between accounts is not represented in the model's input. The defining characteristic of organised fraud is then unavailable to the detector.
2. **Predictable Thresholds**: Fixed numeric limits are trivially evaded. A rule that flags transfers above a stated amount is defeated by transferring slightly less.
3. **No Ring-Level View**: Even when several accounts in a ring are individually flagged, nothing links those alerts together. The analyst sees scattered incidents rather than one coordinated operation.
4. **High False Positive Volume**: Broad rules generate large alert volumes with no prioritisation, so genuinely serious cases compete for attention with routine ones.
5. **Class Imbalance**: Fraudulent transactions form a very small fraction of total volume — 3.33% in the dataset used here. Models trained without explicit imbalance handling optimise accuracy by predicting the majority class and miss the cases that matter. The imbalance is milder at the account level, where a single fraudulent account accounts for many transactions, and the two rates should not be conflated.
6. **Detection Without Workflow**: Much published work stops at producing a score [10], with no built-in path from a flagged account to an assigned, tracked and documented investigation.
7. **Disconnected Tooling**: Detection, alerting and case management typically live in separate systems with no shared data model, forcing manual re-entry between stages.

2.2 Proposed System

The proposed system treats the entire transaction history as a connected network of accounts and layers multiple complementary detection techniques over it. Its key design decisions are:

- **A Transaction Graph as the Core Data Structure**: Accounts are modelled as nodes and transactions as directed, weighted edges in a NetworkX directed graph. Every subsequent analysis operates on this shared representation.
- **A Transparent Rule Engine**: Six business-logic rules derived from standard anti-money-laundering typologies provide an explainable first detection pass that requires no training data and states a clear reason for every flag.
- **Structural Graph Analysis**: PageRank identifies accounts that money concentrates into; degree centrality measures counterparty breadth; Louvain community detection

isolates densely connected clusters; and cycle detection finds closed money loops characteristic of layering.

- **Graph-Aware Feature Engineering**: Eleven transaction-level attributes are combined with six graph-derived attributes into a seventeen-feature vector per account, giving the machine learning models access to relational information they could not otherwise observe.
- **Imbalance-Aware Machine Learning**: Each model family handles the rarity of fraud by its own mechanism. Random Forest is trained with `class_weight="balanced"` and XGBoost with `scale_pos_weight`; Isolation Forest is unsupervised — it never sees a label and is instead given a contamination parameter set to the observed fraud rate of the dataset (Section 7.1.2 explains why an estimate taken from the training partition alone is the sounder choice, and shows that the two give identical results here); the two graph neural networks use a cross-entropy loss weighted by inverse class frequency. These are four distinct mechanisms, not one, and they are set out in Table 7.2.
- **Ring-Level Grouping**: Connected suspicious accounts are grouped into fraud rings using connected-component analysis with community-based splitting, and each member is assigned a role—main, source or withdrawal—reflecting its position in the money flow.
- **An Integrated Investigation Workflow**: Alerts, fraud rings and investigation cases share one data model, so an analyst can move from a flagged account to a closed case without leaving the system.

2.2.1 Advantages of Proposed System

1. **Detects Coordinated Fraud**: Because the graph is the primary data structure, patterns that span multiple accounts are detected directly rather than inferred from scattered individual alerts.
2. **Layered Defence**: A fraudster who evades the fixed thresholds of the rule engine still exposes an abnormal structural position to the graph analysis and the learned models.
3. **Relational Signal Available to the Models**: Six graph-derived measures are placed in the feature vector alongside the transaction-level aggregates, so structural position is available to the learned models rather than invisible to them. Whether those six features improve predictive performance is a separate question that this project does not answer: no ablation comparing models trained with and without them was run, and in the recorded XGBoost importances the graph-derived features are minor con-

tributors (Section 7.2.3). The claim made here is that the information is represented, not that a measured gain has been demonstrated.

4. **Traceable Output**: Every alert records the specific rule violated or the model score that triggered it, and the graph metrics behind a score are readable quantities. The learned models themselves are not equally transparent, and Section 7.3.1 separates the four kinds of explanation the system actually produces.
5. **Prioritised Analyst Attention**: Alerts carry a risk level and a fraud score, so the highest-risk cases surface first instead of competing with routine flags.
6. **Complete Audit Trail**: Every alert and case retains a creation timestamp, current status and final decision, supporting later review and accountability.
7. **Extensible Architecture**: Each detection stage is an independent module reading from and writing to the shared database, so a new rule, metric or model can be added without modifying the stages around it.

Chapter 3

Requirement Analysis

This chapter outlines the complete requirements for the system, covering both hardware and software prerequisites, the functional capabilities the system must deliver, and non-functional constraints including performance, security, reliability and scalability.

3.1 Purpose

The purpose of this system is to detect coordinated financial fraud that transaction-level analysis cannot identify, by modelling the transaction history as a graph and applying rule-based checks, structural graph analysis and machine learning over that representation. The system is intended to be operated by a fraud analyst through a web interface, and to carry every detection through to a documented investigation outcome.

3.2 Overall Description

The system consists of two deployable components:

1. **Backend Service**: A Python FastAPI application exposing a REST API. All data pre-processing, rule evaluation, graph construction, feature engineering, model training and scoring, alert generation and report generation occur in this service.
2. **Frontend Application**: A React single-page application built with Vite and Material UI that communicates with the backend over HTTPS and presents the analyst-facing interface for every module.

3.2.1 Hardware Requirements

Table 3.1: Minimum Hardware Requirements

Component	Minimum	Recommended
CPU	2-core x86-64	4-core x86-64
RAM	4 GB	8 GB
Storage	2 GB free	10 GB free
Display	1366 × 768	1920 × 1080
Network	10 Mbps	100 Mbps

3.2.2 Software Requirements

Table 3.2: Software Requirements

Software	Version	Role
Python	3.10 or later	Backend runtime
FastAPI	0.111	REST API framework
Uvicorn	0.30	ASGI server
SQLAlchemy	2.0	Object-relational mapping
SQLite	stdlib	Relational data store
NetworkX	3.2	Graph construction and analysis
scikit-learn	1.5	Random Forest, Isolation Forest
XGBoost	2.1	Gradient-boosted classifier
PyTorch	2.2 or later	Deep learning runtime for the GNNs
PyTorch Geometric	2.5 or later	GCN and GraphSAGE layers, Elliptic dataset
pandas	2.2	Data manipulation
ReportLab	4.1	PDF report generation
pytest	8.0 or later	Automated test suite
aiosmtpd	1.4 or later	Local SMTP server used by the email tests
React	18	Frontend library
Vite	5.x	Frontend build tool
Material UI	5.x	Component library
Operating System	Windows / Linux / macOS	Development and deployment

The versions are the minimums declared in `backend/requirements*.txt`, which uses `>=` constraints; the exact versions installed for a given run are not pinned (Section 7.2.7).

3.3 Functional Requirements

The system shall fulfil the following functional requirements:

- Authenticate users with a username and password and issue a session token.
- Store, list, search and filter customer accounts and transaction records.
- Detect and report data-quality issues including missing values, duplicate records and invalid account references.
- Evaluate every transaction against six business-logic fraud rules and mark matching transactions as suspicious.
- Construct a directed transaction graph from account and transaction data.
- Compute PageRank, degree centrality, community membership and cycle membership for every account.
- Generate a seventeen-attribute feature vector for every account.
- Train Random Forest, XGBoost and Isolation Forest models and persist the best-performing model.
- Train Graph Convolutional Network and GraphSAGE models directly on the account graph, and evaluate them on an independent public benchmark.
- Assign a fraud probability score to every account, and set its risk level from the rule engine alone.
- Generate fraud alerts from rule violations and from accounts exceeding a configurable score threshold.
- Group connected suspicious accounts into fraud rings and assign member roles.
- Allow an analyst to place an account under review, confirm it as fraudulent or clear it, and record every such decision with its author, timestamp and reason.
- Raise a high-priority alert and push a real-time notification when an account an analyst has confirmed as fraudulent sends a further transaction.
- Notify the holder of a confirmed fraudulent account by email, logging every delivery attempt and its outcome.
- Create and manage investigation cases with notes, an assigned officer and a status lifecycle.
- Present aggregate statistics and charts on a dashboard.
- Export a PDF summary report and CSV extracts of key tables.

- Provide a scripted demonstration dataset that exercises all six rules end to end without disturbing the working data.

3.3.1 Python and FastAPI

Python is the primary language for the backend. It was chosen for the maturity of its data and machine learning ecosystem—pandas, scikit-learn [14], XGBoost [4] and NetworkX [13] are all first-class Python libraries—which allows the entire detection pipeline to be written in a single language without inter-process data exchange.

FastAPI [16] provides the HTTP layer. It was selected for its native async/await support, automatic OpenAPI documentation, Pydantic-based request validation, and its dependency injection system, which allows authentication to be applied declaratively to every protected route rather than repeated in each handler.

3.3.2 NetworkX and Graph Algorithms

NetworkX [13] provides the graph data structure and the algorithm implementations that form the core of the project. A directed graph is built with accounts as nodes and transactions as weighted edges, and four classes of analysis are run over it: PageRank for structural importance, degree centrality for counterparty breadth, Louvain community detection for cluster isolation, and cycle detection for closed money loops. NetworkX was chosen over a dedicated graph database because the graph is rebuilt from relational data on demand, requires no separate server process, and fits comfortably in memory at the scale of this project. The PageRank implementation additionally requires SciPy for its sparse-matrix solver.

3.3.3 PyTorch Geometric and Graph Neural Networks

NetworkX supplies structure as explicit, hand-chosen measurements. PyTorch Geometric [15] supplies the complementary approach: a Graph Convolutional Network and a GraphSAGE model that learn their own representation of an account from the accounts it transacts with, through two rounds of message passing. It was chosen because it provides reference implementations of both architectures [6, 7] over a common data format, and because it ships a loader for the Elliptic Bitcoin dataset [8], which the project uses as an independent benchmark. PyTorch runs on the CPU here; no GPU is required at this scale. The dependency is kept in a separate requirements file because it is a large download, and the application degrades gracefully when it is absent: the graph-neural-network pages report that the dependency is missing rather than failing.

3.3.4 SQLite and SQLAlchemy

SQLite is used as the relational data store. It is an embedded database requiring no separate server process, which keeps deployment simple, and it is well suited to the read-heavy, single-writer access pattern of this system.

All database access is mediated by the SQLAlchemy ORM [17]. Application code queries Python model classes rather than writing SQL strings, and SQLAlchemy generates the dialect-appropriate SQL. This keeps the application code largely independent of the specific engine: the model classes and queries would not need rewriting to move to a production engine such as PostgreSQL. The ORM reduces, but does not eliminate, the work involved. A migration would still require installing and configuring the appropriate driver, creating the schema on the new engine and migrating existing data, reviewing the handful of places where SQLite's permissive typing differs from a strictly typed engine, and re-running the test suite against it. The ORM also removes a class of SQL injection risk, since parameters are bound rather than interpolated.

3.3.5 React and Material UI

The frontend is a React single-page application built with Vite. React was chosen for its component model, which maps cleanly onto the system's eighteen distinct pages, and for the maturity of its ecosystem. Vite provides fast development builds and an optimised production bundle. Material UI supplies the component library—tables, dialogs, cards and form controls—giving a consistent interface without hand-written styling, and Recharts renders the dashboard visualisations. Axios handles HTTP communication, with an interceptor that attaches the authentication token to every outgoing request.

3.3.6 Fraud Rule Specification

Table 3.3 states the six rules exactly as implemented in `app/ml/rules.py`. All monetary thresholds are in Indian rupees (Rs.). They were chosen to produce a workable alert volume on the synthetic dataset and are demonstration values: they are modelled on common anti-money-laundering typologies but are not statutory reporting thresholds, and no claim of regulatory compliance is made for them.

Indian reporting obligations are not a single figure that R1 could be compared against. Under the Prevention of Money Laundering (Maintenance of Records) Rules, a reporting entity files a cash transaction report for cash transactions above Rs. 10,00,000, a cross-border wire transfer report above Rs. 5,00,000, and a suspicious transaction report with no monetary threshold at all — suspicion alone is the trigger, whether or not cash is involved [18]. R1 is closer in spirit to the last of these than to the first: it raises suspicion, it does not discharge a reporting duty, and its Rs. 1,00,000 figure applies to every trans-

action type in the dataset rather than to cash alone. A deployment would have to set each threshold against the specific obligation and channel it serves.

Table 3.3: Implemented Fraud Rules

ID	Internal name	Detection logic	Threshold / window	Pattern identified
R1	`large_amount`	A single transaction whose amount exceeds the configured large-amount threshold	amount $>$ Rs. 1,00,000	A large transfer made in one movement
R2	`rapid_transactions`	One sender making an unusual burst of transfers in a short window	$>$ 5 transactions in 10 minutes	Rapid dispersal before an account can be frozen
R3	`new_account_large`	A recently opened account receiving a large credit	account age $<$ 30 days and amount $\geq$ Rs. 50,000	A freshly opened mule account being funded
R4	`fan_in`	Many distinct senders paying into one account within a window (M $\rightarrow$ 1)	$\geq$ 8 distinct senders in 1 hour	Collection of funds into a consolidating account
R5	`fan_out`	One account paying many distinct receivers within a window (1 $\rightarrow$ M)	$\geq$ 8 distinct receivers in 1 hour	Dispersal of funds across cash-out accounts
R6	`circular`	Money returning to its origin through two intermediate accounts (A $\rightarrow$ B $\rightarrow$ C $\rightarrow$ A)	each hop within 48 hours of the previous hop	Layering through shell accounts

For every match the engine marks the transaction `status = "suspicious"`, writes a row to the `rule_hits` table recording which rule fired on which transaction, and raises the risk level of the accounts involved to Medium, or to High where two or more distinct rules have fired. The rule engine is the only component that writes an account's risk level; model scores never alter it. R6 examines three-hop loops only, and because each hop is timed against the one before it, a complete loop may span up to 96 hours. Two-account loops, and loops irrespective of timing, are represented instead by the `in_cycle` graph feature (Table 4.2).

3.4 Non-Functional Requirements

3.4.1 Performance Requirements

- Response time for simple read endpoints must remain under 500 ms on recommended hardware.
- Graph construction and analysis over a 5,000-node graph must complete in seconds.
- The rule engine must process the full 80,000-transaction dataset within two minutes.
- Account and transaction listings must be paginated so that response size remains bounded regardless of dataset size.

3.4.2 Security Requirements

- All endpoints handling protected application data require a valid bearer token, with two explicitly documented exceptions: the health check at `GET /` and the system-information endpoint at `GET /api/system/info`, which returns a configuration flag and no application data. The login endpoint itself is necessarily unauthenticated. Accounts, transactions, alerts, rings, cases, investigation records, dashboards, reports and all model and graph analytics are protected; Section 4.7.1 lists the dependencies that enforce this.
- Opening an account's investigation record, changing its fraud status, retrying an account-holder email, receiving fraud notifications and reading the model evaluation analytics are restricted to the administrator and analyst roles.
- The application refuses to start in a production configuration whose signing key, administrator password or mail transport is left at an insecure default.
- Passwords are stored as salted hashes and never in plaintext.
- Cross-origin requests are accepted only from an explicitly configured allow-list.
- Secrets such as the token signing key and the administrator credentials are read from environment variables, never from source control.
- All database access passes through the ORM with bound parameters, eliminating string-interpolated SQL.

3.4.3 Reliability and Scalability

- Re-running a detection stage must not duplicate alerts already recorded.

- Computed graph metrics are persisted so that expensive analysis runs once and is then read many times by the stages that depend on it.
- If the database is found empty at startup, the baseline dataset is seeded automatically so the application is never left unusable.
- The architecture must permit SQLite to be replaced by a production-grade database without changes to the API contract or the query code.
- Every alert and case retains a creation timestamp and status to support audit.

Chapter 4

Design and Implementation

This chapter presents the architectural and implementation details of Financial Fraud Detection Using Graph Machine Learning. It covers the layered system design, the detection pipeline, data flow across the processing stages, database schemas, the React frontend architecture, and screenshots of the user interface.

4.1 Overall Design

The system follows a **layered architecture** with clear separation between the client tier, the authentication layer, the API gateway, the detection pipeline, the backend services and persistence. Every request from the single-page application carries a JWT bearer token, which the authentication layer validates before the request reaches a router; the only exceptions are the login endpoint, the health check and a system-information endpoint that returns a configuration flag and no data. Each layer is independently testable and replaceable.

4.2 System Design

4.2.1 Database Schema

All application state is held in a single relational database accessed exclusively through the SQLAlchemy ORM. No module issues raw SQL, so the schema and the queries are expressed once, in the model classes, rather than scattered through the codebase as literal statements.

Row counts are those of the working database at the time of writing: the seeded dataset contributes 5,000 accounts and 80,000 transactions, and the remainder are the accounts and transactions added by hand through the interface while the deployed instance was tested and demonstrated (Section 7.1.1). The `alerts` and `rule_hits` counts likewise describe that working database; a fresh run over the seeded data alone produces 2,979 rule hits and, at the default alert threshold of 0.80, 4,134 alerts. Two artefacts of the pipeline are

deliberately *not* database tables. The seventeen-column feature matrix is written to `data/processed/features.csv`, and the assembled fraud rings are cached to `saved_models/rings.json`; both are derived outputs that are regenerated whenever the pipeline is re-run, so persisting them as relations would duplicate state that the pipeline already owns. Relationships between tables are carried by the business keys (`account_uid`, `txn_uid`, `alert_uid`) rather than by declared foreign-key constraints; the one exception is `notification_reads`, which declares a foreign key on `fraud_notifications.id`. This is a deliberate simplification for a single-process SQLite deployment and is recorded among the limitations in Chapter 7.

Table 4.1: Backend Persistence Schema

Table	Rows	Key Columns
`accounts`	5,003	`id` (PK), `account_uid` (unique), `customer_name`, `email`, `phone`, `account_type`, `risk_level`, `fraud_score`, `is_fraud`, `review_status`, `created_at`
`transactions`	80,004	`id` (PK), `txn_uid` (unique), `sender_uid`, `receiver_uid`, `amount`, `timestamp`, `txn_type`, `status`, `is_fraud`, `fraud_score`
`graph_metrics`	5,000	`account_uid`, `in_degree`, `out_degree`, `total_sent`, `total_received`, `degree_centrality`, `pagerank`, `component_id`, `component_size`, `community_id`, `community_size`, `in_cycle`
`alerts`	4,212	`alert_uid`, `transaction_uid`, `account_uid`, `reason`, `fraud_score`, `risk_level`, `status`
`rule_hits`	2,981	`txn_uid`, `rule` — one row per rule fired on a transaction
`cases`	variable	`case_uid`, `alert_uid`, `account_uid`, `assigned_to`, `notes`, `status`, `final_decision`
`account_status_events`	variable	`account_uid`, `from_status`, `to_status`, `changed_by`, `transaction_uid`, `note`, `created_at`
`fraud_notifications`	variable	`kind`, `priority`, `account_uid`, `transaction_uid`, `amount`, `risk_score`, `message`
`notification_reads`	variable	`notification_id` (FK), `user_id`, `read_at`
`email_deliveries`	variable	`kind`, `account_uid`, `status_event_id`, `recipient_masked`, `status`, `attempts`, `last_error`
`users`	1	`username` (unique), `hashed_password`, `role`
`schema_migrations`	3	`version`, `applied_at` — records the start-up schema upgrades already applied

4.2.2 Backend Architecture

The backend is structured as a Python package (`backend/app/`) with clearly separated sub-packages:

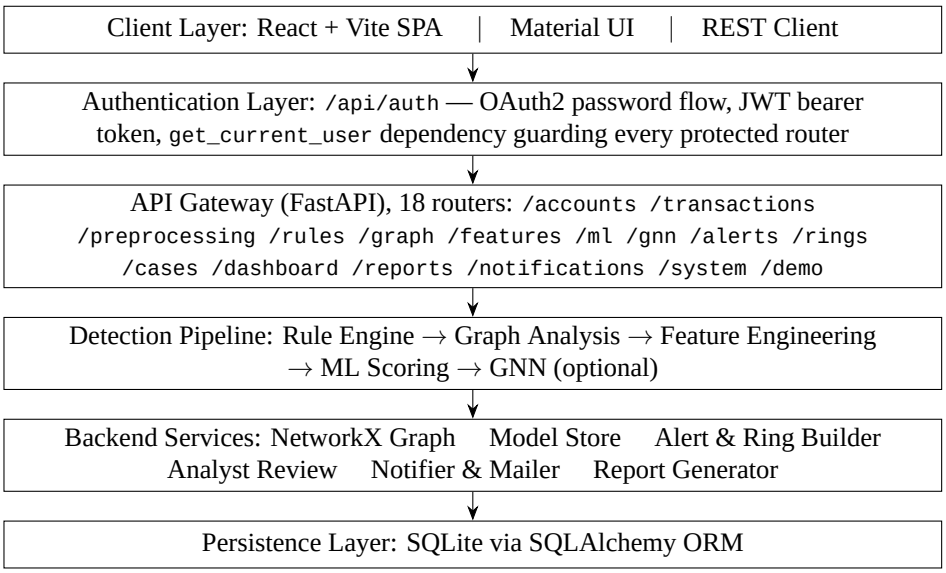


Figure 4.1: High-Level System Architecture

- `api/` — eighteen FastAPI routers, one per resource: `auth`, `accounts`, `transactions`, `preprocessing`, `rules`, `graph`, `features`, `ml`, `gnn`, `alerts`, `rings`, `cases`, `dashboard`, `reports`, `investigation` (mounted under the `/api/accounts` prefix), `notifications`, `system` and `demo`.
- `ml/` — the detection modules: `rules.py` (rule engine), `graph_build.py` and `graph_analysis.py` (graph metrics), `features.py` (feature engineering), `baseline.py` (classical model training and scoring), `gnn.py` (GCN and GraphSAGE), `alerts.py` and `fraud_ring.py`.
- `services/` — the response workflow: `fraud_review.py` (analyst decisions and their audit trail), `notifications.py` (real-time alerting) and `mailer.py` (account-holder email).
- `demo/` — the scripted demonstration scenario as pure data, with no database access, so both the planting script and the application can import it.
- `models/` — SQLAlchemy declarative models mirroring Table 4.1, and the Pydantic schemas used for request and response validation.
- Root modules — `main.py` holds the application factory and CORS configuration, `config.py` the environment-driven settings, `database.py` the engine and session factory, `deps.py` the authentication and role dependencies, `migrations.py` the start-up schema upgrade, and `core/security.py` the JWT issuance and password hashing.

4.2.3 Detection Pipeline Orchestration

The detection stages are exposed as independent endpoints rather than as a single monolithic job, because each stage has a different cost profile and the analyst needs to be able to re-run one stage without repeating the others. The stages are ordered by dependency: the rule engine reads only the transaction table and can run first; graph analysis builds the NetworkX `DiGraph` and persists per-account metrics; feature engineering joins the transaction aggregates with those metrics; model scoring consumes the resulting feature matrix; and the graph neural networks train on the graph and the same feature matrix together. Each endpoint writes its output to the database and returns a summary, so progress survives a page reload and the frontend remains stateless.

The rule engine is also the only writer of an account's risk level. A model score never raises it, which keeps the one field an analyst reads at a glance traceable to a stated rule. A separate `onboarding_risk_category`, set by hand at customer onboarding, is held apart from it and feeds neither the rules nor the models.

4.2.4 Analyst Review and Response

Detection output stops short of a verdict. An account the rules or the models find suspicious is marked *Suspicious*; only an analyst can move it to *Under Review*, *Confirmed Fraud* or back to cleared, and clearing requires a written reason. Three things follow from a confirmation, and the design keeps them consistent:

- **An audit trail.** Every status change writes an `account_status_events` row recording the analyst, the time, the note and the transaction the decision refers to.
- **An email to the account holder**, queued in the same database transaction as the confirmation, so a confirmation always has exactly one email record — marked as skipped when the account has no usable address. Repeating a confirmation changes nothing and sends nothing, and clearing an account sends no email at all. Delivery happens after the status is saved, so a mail failure never undoes it; each attempt is logged and can be retried.
- **Real-time alerting.** When a confirmed account sends a further transaction, a high-priority alert and a notification are stored and every open notification stream is woken. The database is the source of truth, so a notification survives a refresh, a new login or a restart, and a client that was disconnected catches up on reconnecting.

4.2.5 Demonstration Scenario

A scripted mule-network laundering operation is bundled for demonstrations. Twenty-two accounts and twenty-two transactions, all identified by a `DEMO-` prefix, tell a four-stage

story chosen so that the complete rule set fires in sequence:

1. **Collection** — nine feeder accounts pay a mule account opened six days earlier, all within twenty-five minutes (triggers R3 and R4).
2. **Consolidation** — the mule forwards almost the whole balance to a collector in one transfer (R1).
3. **Layering** — collector → shell 1 → shell 2 → collector, each hop deliberately under the reporting threshold (R6).
4. **Cash-out** — nine transfers just under the threshold to nine cash-out accounts within eight minutes (R2 and R5).

The scenario is defined as pure data with no database access, so the planting script and the application share one definition. It can be planted into a copy of the working data or served as a standalone database containing nothing else, which keeps a demonstration from disturbing the real dataset.

4.2.6 Frontend Framework (React)

The React application is built with Vite and follows a hooks-and-service-module pattern:

- **API modules** (`api/accounts.js`, `api/graph.js`, `api/ml.js`, `api/alerts.js`, `api/cases.js`, `api/dashboard.js`, `api/reports.js` and their siblings) wrap Axios calls and attach the bearer token through a shared interceptor, so no component constructs a URL or handles a header directly.
- **Pages** (`Login`, `Dashboard`, `Accounts`, `AccountDetail`, `Transactions`, `DataQuality`, `RuleDetection`, `GraphAnalysis`, `Features`, `MLScoring`, `GNN`, `ModelAnalytics`, `Alerts`, `Notifications`, `FraudRings`, `Cases`, `Reports`, `DemoScenario`) hold their own data-fetching state with `useState` and `useEffect` and render Material UI components.
- **Shared components** provide the application shell, the protected-route wrapper that redirects unauthenticated users to the login page, the money-flow network view, and the Recharts-based chart wrappers used on the dashboard. The shell hides the Model Analytics and Notifications pages from users without an analyst role.

4.3 Use Case Diagram

Figure 4.2 shows the principal use cases. The application defines three roles — `admin`, `analyst` and `viewer` (`ROLES` in `scripts/create_user.py`) — but only two permission sets: `FRAUD_ANALYST_ROLES` in `app/deps.py` treats `admin` and `analyst` identically, so they

are drawn as a single actor. Every router requires a signed-in user through `get_current_user`; a second dependency, `require_fraud_analyst`, restricts the shaded use cases to that actor. It guards the whole investigation router — the account investigation record and the review action that marks an account under review, confirms it or clears it, together with the retry of a failed holder email, which is omitted from the diagram for brevity — as well as the notification endpoints and the graph-neural-network evaluation behind the Model Analytics page.

A `viewer` may therefore browse accounts, transactions and graph metrics, run every detection stage including model training, review alerts and fraud rings, and work investigation cases and reports. A viewer cannot open an account's investigation view, change its review status, receive fraud notifications or read the model analytics: the interface hides the Model Analytics and Notifications pages from that role and shows a notice in place of the investigation view, and the API answers 403 if any of them is requested directly. Because the analyst can do everything a viewer can, the Fraud Analyst / Administrator actor is drawn as a specialisation of Viewer, inheriting all of its use cases.

Confirming and clearing are drawn as separate use cases because they behave differently. Every effective confirmation queues exactly one account-holder email in the same database transaction (Section 4.2.4), so *Confirm fraud* «include»s *Notify account holder by email*; whether the queued email is actually delivered depends on the address on file, as the attached note records. Clearing requires a reason and sends nothing, so it has no relationship with the notification. The Account Holder is not a user of the system and initiates none of these actions: they appear only as the secondary actor who receives the notice.

4.4 Context and Data Flow Diagrams

Figure 4.3 shows the system boundary. Figure 4.4 decomposes it into the major processes and the data stores they read and write. The detection pipeline itself is shown separately, as a processing flowchart, in Figure 4.5.

4.5 Detection Pipeline Flow

Figure 4.5 shows the order in which the detection stages run and what each one hands to the next. The solid path is the operational pipeline: alert generation reads exactly two upstream signals — transactions the rule engine marked `suspicious`, and accounts whose `fraud_score` the baseline models wrote — so everything downstream of it depends only on the rule engine and the classical models.

The graph neural networks sit on the dashed branch. They read the same feature matrix and the same graph, but nothing downstream consumes their output: the `/api/gnn` router

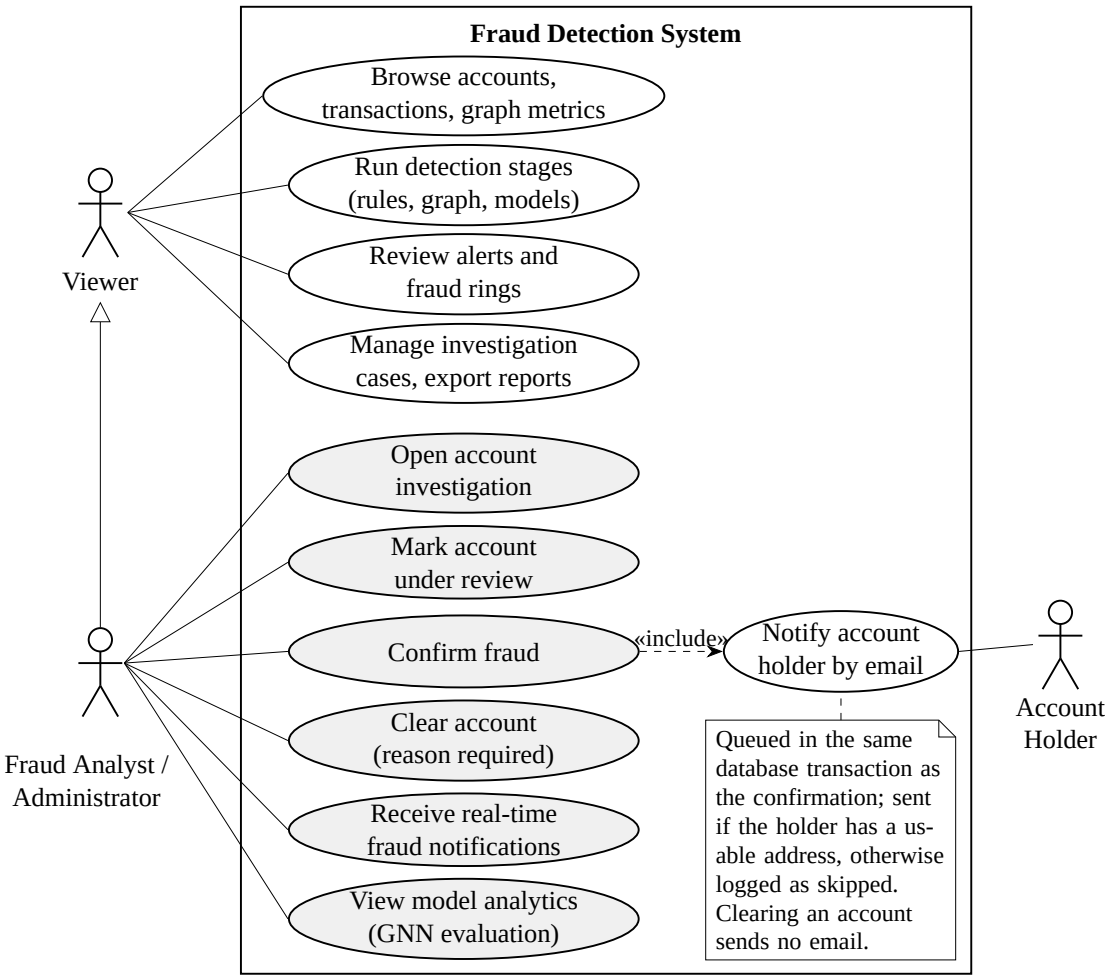


Figure 4.2: Use Case Diagram

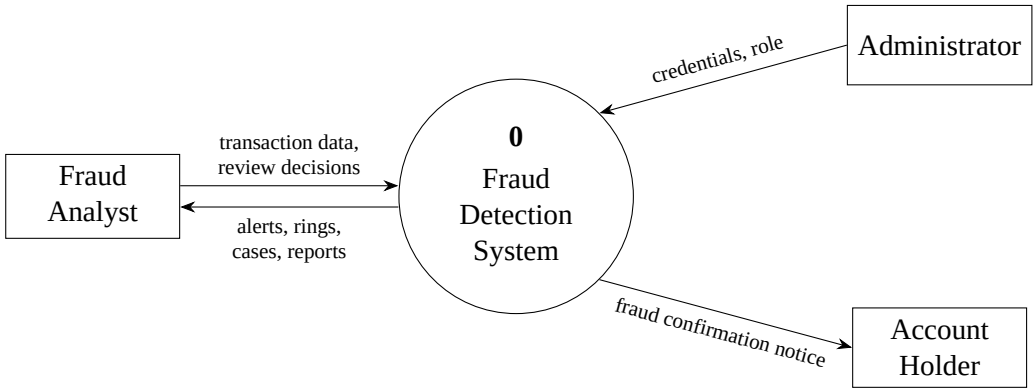


Figure 4.3: Context-Level Data Flow Diagram (Level 0)

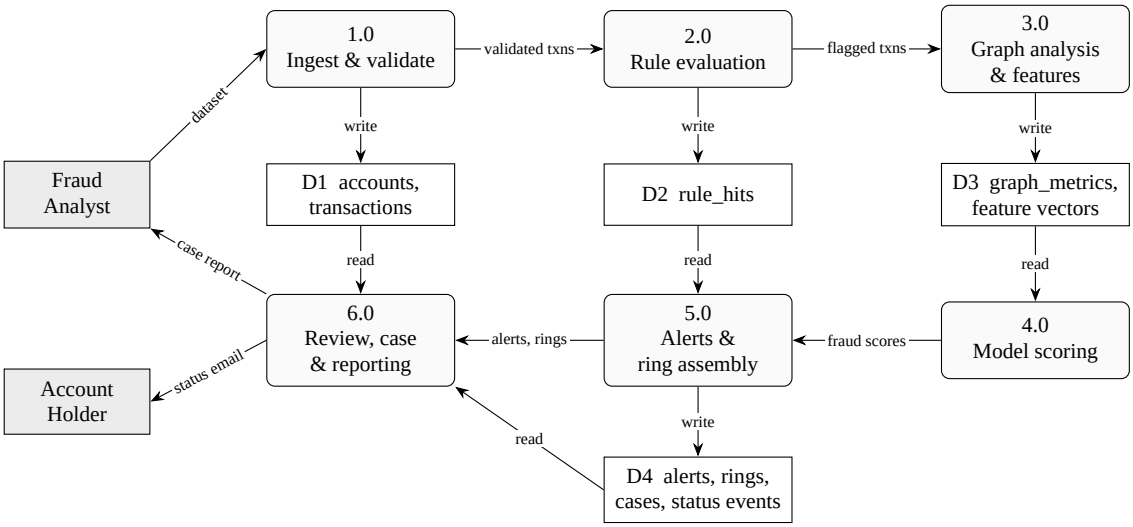


Figure 4.4: Level-1 Data Flow Diagram

exposes only `train`, `status` and `evaluation`, and `alerts.generate()` never reads a GNN score. Training is triggered separately by the analyst, requires PyTorch and PyTorch Geometric, and is unavailable on the free-tier deployment, which does not carry those packages. A complete detection run therefore produces alerts, rings and cases whether or not the graph neural networks have ever been trained.

4.6 Database Design

4.6.1 Entity Relationship Diagram

Figure 4.6 shows the core entities and their relationships, taken from the SQLAlchemy model classes in `app/models/`. Underlined attributes are the surrogate integer primary keys; `UQ` marks the unique business key each entity is actually referenced by. The relationships shown are those the application maintains in code through those business keys (`account_uid`, `txn_uid`, `alert_uid`); only `notification_reads.notification_id`, marked `FK`, is a foreign key declared to the database. Section 7.3.2 records that distinction as a limitation.

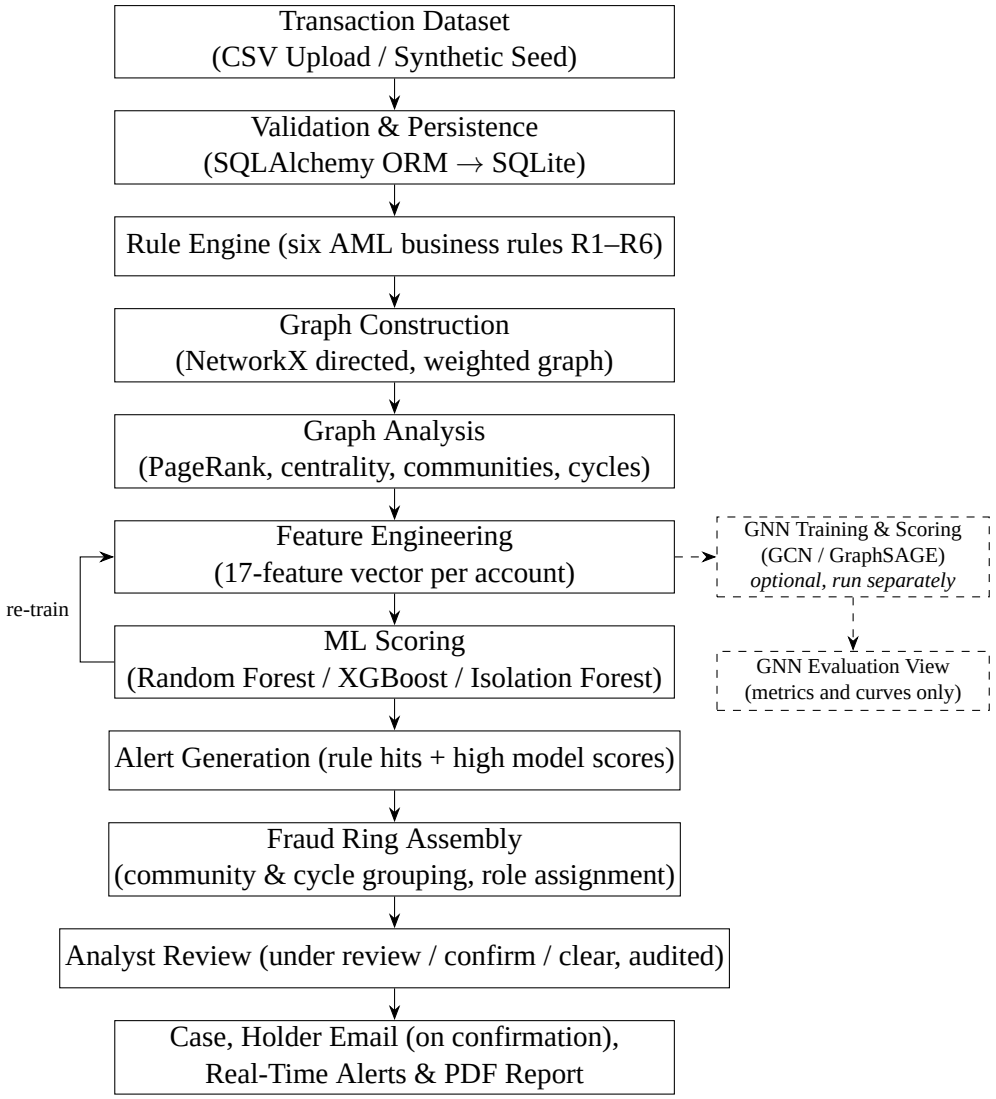
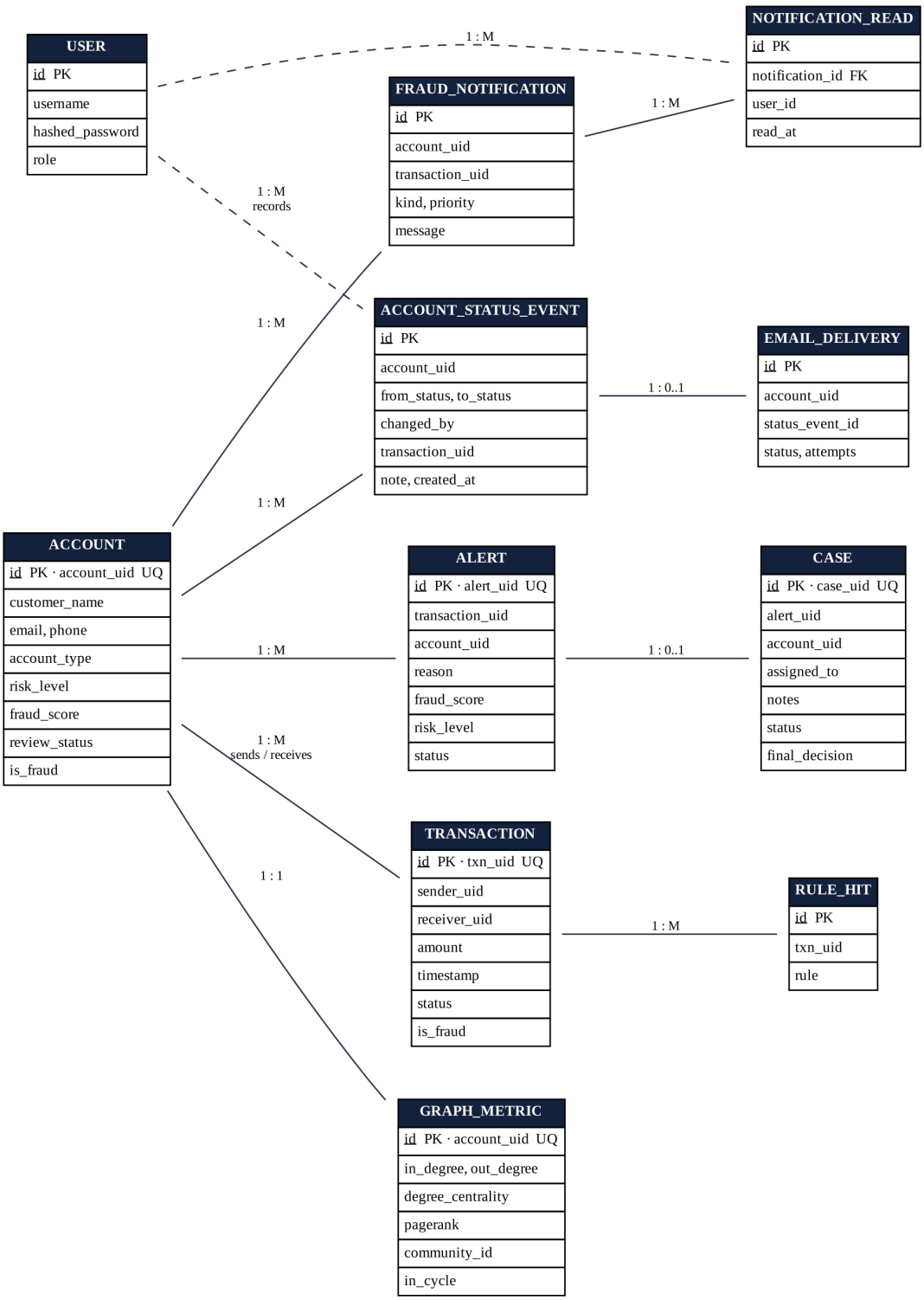


Figure 4.5: Detection Pipeline Data Flow Diagram



Primary keys are surrogate integers (id). Entities are joined on the unique business keys (account_uid, txn_uid, alert_uid); only notification_reads declares a database foreign key.

Figure 4.6: Entity Relationship Diagram

4.6.2 Table Definitions

Accounts Table

Listing 4.1: `accounts` — schema as created by the ORM

```
CREATE TABLE accounts (
    id                      INTEGER NOT NULL,
    account_uid             VARCHAR NOT NULL,   -- business key, e.g. ACC00001
    customer_name           VARCHAR NOT NULL,
    email                   VARCHAR,
    phone                   VARCHAR,
    account_type            VARCHAR,            -- Savings | Current | Business | Wallet
    created_at              DATETIME,
    risk_level              VARCHAR,            -- Low | Medium | High (rule engine only)
    fraud_score             FLOAT,              -- model output, 0..1
    is_fraud                INTEGER,            -- ground-truth label from the dataset
    is_active               INTEGER,
    review_status           VARCHAR,            -- Under Review | Confirmed Fraud | NULL
    review_updated_at       DATETIME,
    fraud_confirmed_by      VARCHAR,
    fraud_confirmed_at      DATETIME,
    onboarding_risk_category VARCHAR,           -- manual KYC category, used by nothing else
    PRIMARY KEY (id)
);
CREATE UNIQUE INDEX ix_accounts_account_uid   ON accounts (account_uid);
CREATE INDEX        ix_accounts_review_status ON accounts (review_status);
```

Transactions Table

Listing 4.2: `transactions` — schema as created by the ORM

```
CREATE TABLE transactions (
    id              INTEGER NOT NULL,
    txn_uid         VARCHAR NOT NULL,   -- business key, e.g. TXN0000001
    sender_uid      VARCHAR NOT NULL,   -- -> accounts.account_uid
    receiver_uid    VARCHAR NOT NULL,   -- -> accounts.account_uid
    amount          FLOAT   NOT NULL,
    timestamp       DATETIME,
    txn_type        VARCHAR,            -- transfer | payment | withdrawal
    status          VARCHAR,            -- normal | suspicious (set by the rule engine)
    is_fraud        INTEGER,            -- ground-truth label from the dataset
    fraud_score     FLOAT,
    idempotency_key VARCHAR,            -- guards against duplicate submission
    PRIMARY KEY (id)
);
CREATE UNIQUE INDEX ix_transactions_txn_uid      ON transactions (txn_uid);
CREATE INDEX        ix_transactions_sender_uid   ON transactions (sender_uid);
CREATE INDEX        ix_transactions_receiver_uid ON transactions (receiver_uid);
```

Alert Record Structure

A representative row from the `alerts` table, serialised as JSON by the API. The values are taken from an actual alert raised during a pipeline run; a `null` `transaction_uid` indicates an account-level alert raised from a model score rather than from a single transaction. The score threshold is chosen when alerts are generated (0.50–0.99, default 0.80); this alert was raised in a run that used 0.70.

Listing 4.3: Alert record as returned by the API (valid JSON)

```
{
    "alert_uid":       "ALT000001",
    "transaction_uid": null,
    "account_uid":     "ACC00003",
    "reason":          "ML model: account fraud score 1.00 >= 0.70",
    "fraud_score":     0.9999,
    "risk_level":      "High",
    "status":          "Pending",
    "created_at":      "2026-07-19T17:24:39Z"
}
```

Account Feature Vector

Table 4.2: Account Feature Vector — the seventeen columns of `FEATURE_COLS`

Field	Type	Description
Transaction-level features (11)		
`total_sent`	float	Sum of all outgoing amounts
`total_received`	float	Sum of all incoming amounts
`txn_count_sent`	int	Number of outgoing transactions
`txn_count_received`	int	Number of incoming transactions
`unique_receivers`	int	Distinct accounts paid
`unique_senders`	int	Distinct accounts paid by
`avg_sent_amount`	float	Mean outgoing transaction amount
`avg_received_amount`	float	Mean incoming transaction amount
`max_sent_amount`	float	Largest single outgoing transaction
`max_received_amount`	float	Largest single incoming transaction
`txn_per_active_day`	float	Transactions per day on which the account was active
Graph-derived features (6)		
`in_degree`	int	Number of distinct incoming counterparties
`out_degree`	int	Number of distinct outgoing counterparties
`degree_centrality`	float	Normalised total degree in the account graph
`pagerank`	float	PageRank score [1]
`in_cycle`	int	1 if the account lies on a directed cycle of length two or three
`community_size`	int	Size of the account's Louvain community [2]

4.7 Security Design

Every control described here was read from the implementation; the section separates what is built from what is not.

4.7.1 Authentication and Authorisation

Login posts an OAuth2 password form to `/api/auth/login`. The submitted password is verified against a bcrypt hash held in the `users` table, and on success a JSON Web Token is issued, signed with HS256, carrying the username as its subject and expiring after 60

minutes. Protected routes depend on `get_current_user`, which decodes the token, rejects an invalid or expired signature with 401, and loads the user.

A second dependency, `require_fraud_analyst`, restricts the sensitive routes to accounts whose role is `admin` or `analyst` and returns 403 otherwise. It is applied to the whole investigation router — an account's investigation record, the review action that marks it under review, confirms it or clears it, and the retry of a failed holder email — to every notification endpoint, and to the graph-neural-network evaluation that backs the Model Analytics page. Every other router depends on `get_current_user` alone and is open to the `viewer` role (Section 4.3). The notification stream cannot send an `Authorization` header, so it authenticates by token in the query string and the server closes the stream when that token expires.

Two endpoints are deliberately unauthenticated: `GET /`, a health check returning a fixed status string, and `GET /api/system/info`, which reports whether the instance is running in demonstration mode so the client can hide the training controls. Neither reads the database nor returns any account data.

4.7.2 Configuration and Transport

Secrets — the token signing key, the administrator password and the SMTP credentials — are read only from environment variables or a local `.env` file. Pydantic is configured with `hide_input_in_errors`, so secret values do not appear in validation errors or deployment logs. Unless `APP_ENV` is explicitly set to `development`, the application refuses to start if the signing key is a published placeholder, shorter than 32 characters or too repetitive to be random; if the administrator password is the published default or shorter than 12 characters; or if the mail configuration would transmit its password unencrypted. A deployment that omits a variable therefore fails loudly rather than running insecurely.

Cross-origin requests are accepted only from explicitly configured origins, each normalised to `scheme://host:port` before comparison. HTTPS is terminated by the hosting platforms; the application does not implement transport security itself. All database access passes through the ORM with bound parameters.

4.7.3 Audit Trail and Account-Holder Notification

Every change to an account's fraud status writes an `account_status_events` row recording the analyst, the timestamp, the note and the transaction the decision refers to. Each email attempt is logged individually in `email_deliveries` with its outcome.

Confirming an account as fraudulent queues a notice to the account holder. **This is a demonstration feature, and it would be inappropriate as an automatic action in a real institution**: notifying a suspect that they have been identified can defeat the investigation, and in several jurisdictions disclosure is restricted while a suspicious activity

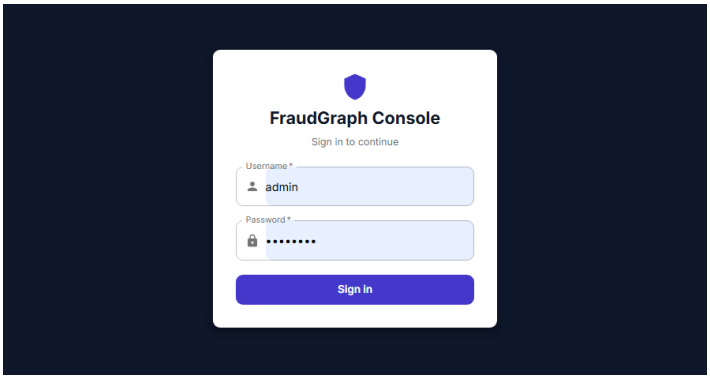
report is pending. In an operational deployment the notice would have to be gated behind an explicit release step authorised by the compliance function, and the system would need to distinguish between a confirmed-fraud victim, who should be told promptly, and a suspected perpetrator, who should not. The implementation here sends to the address on the account without making that distinction.

4.7.4 Controls Not Implemented

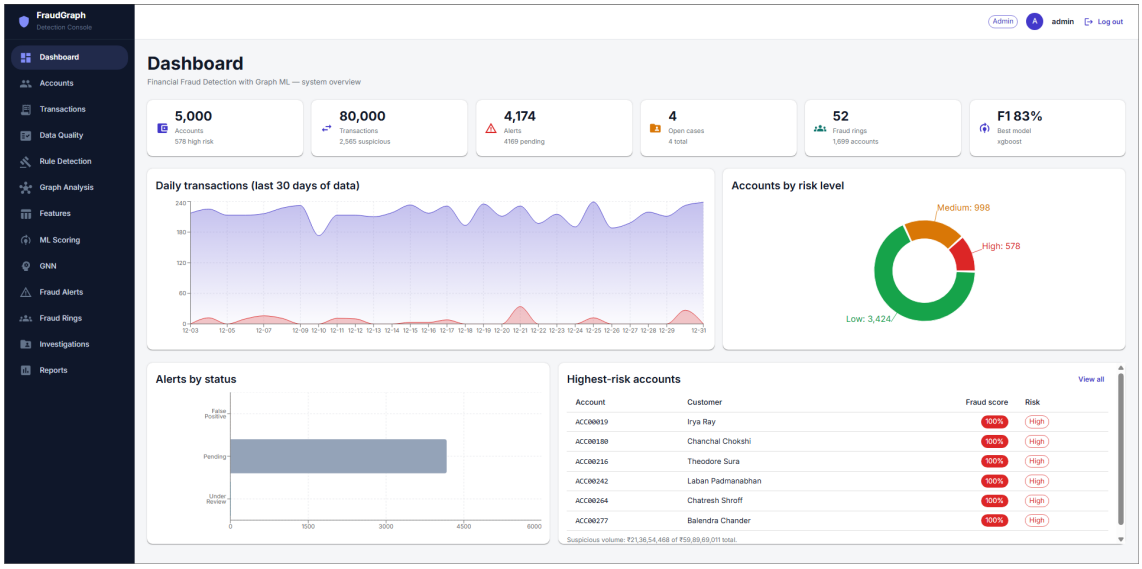
The following are absent from the current build and are listed so that the security posture is not overstated: encryption of data at rest (the SQLite file is stored unencrypted); refresh tokens and server-side token revocation (a token is valid until it expires); rate limiting or lockout on repeated failed logins; and multi-factor authentication. No claim of compliance with any banking-sector standard is made or implied.

4.8 Screenshots of User Interface

The following figures illustrate key screens of the analyst web interface. All are static captures of the running application.

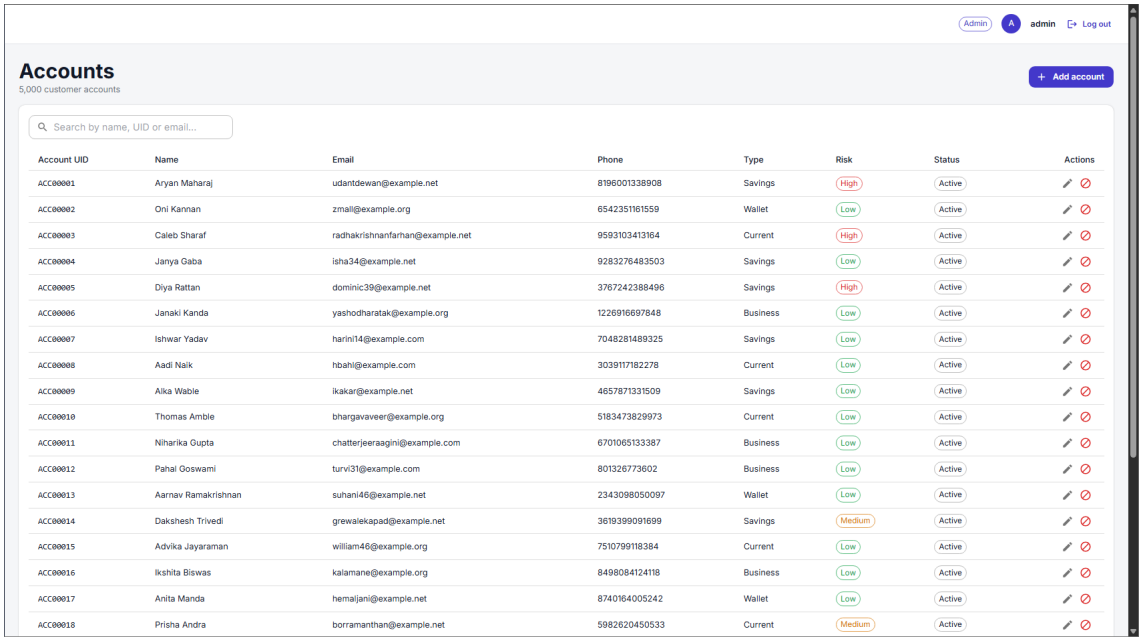


(a) Login Screen — JWT Authentication

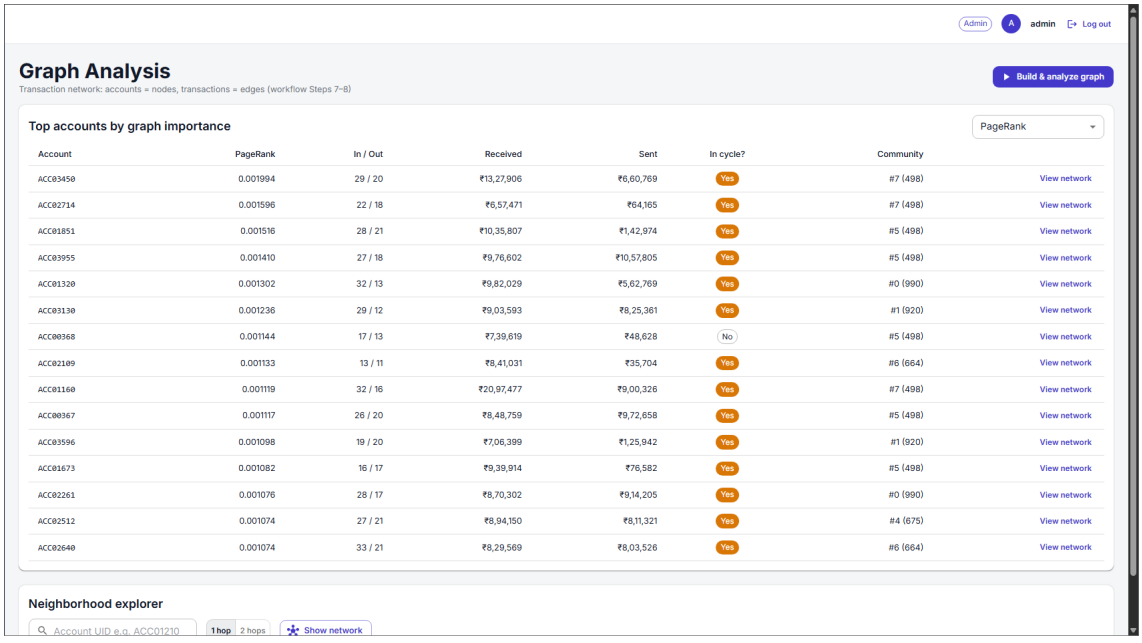


(b) Dashboard — Portfolio Summary and Risk Distribution

Figure 4.7: User Interface — Authentication and Dashboard



(a) Accounts View — Searchable Account Register with Risk Levels



(b) Graph Analysis — accounts ranked by graph importance, with a neighbourhood explorer for a selected account

Figure 4.8: User Interface — Accounts and Graph Analysis

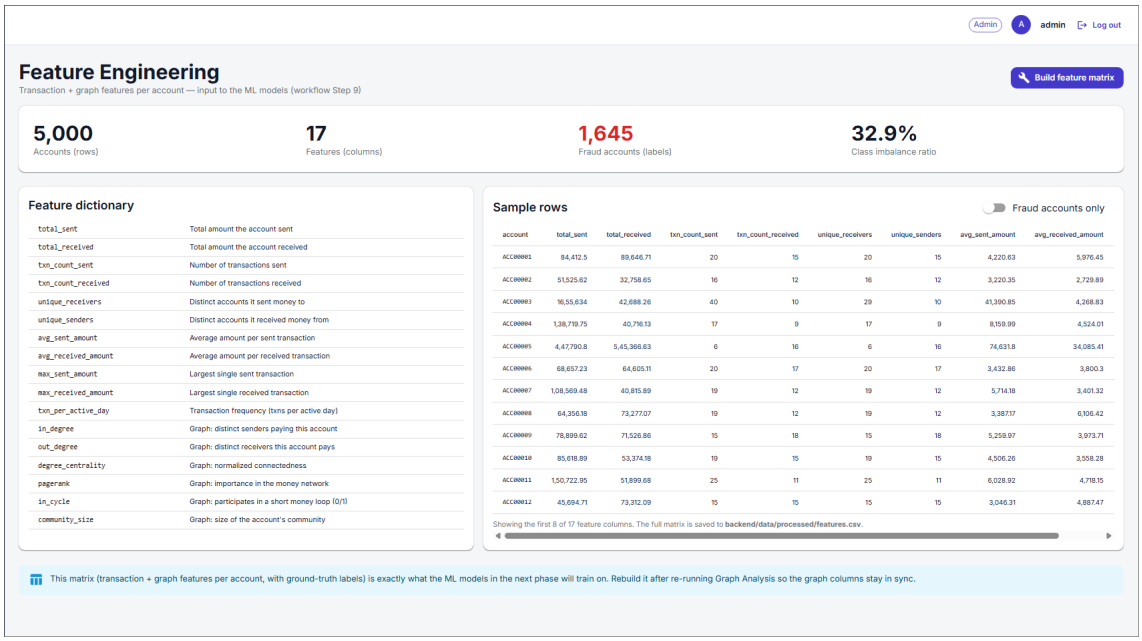


Figure 4.9: Feature Engineering View — Per-Account Feature Matrix and Model Scores

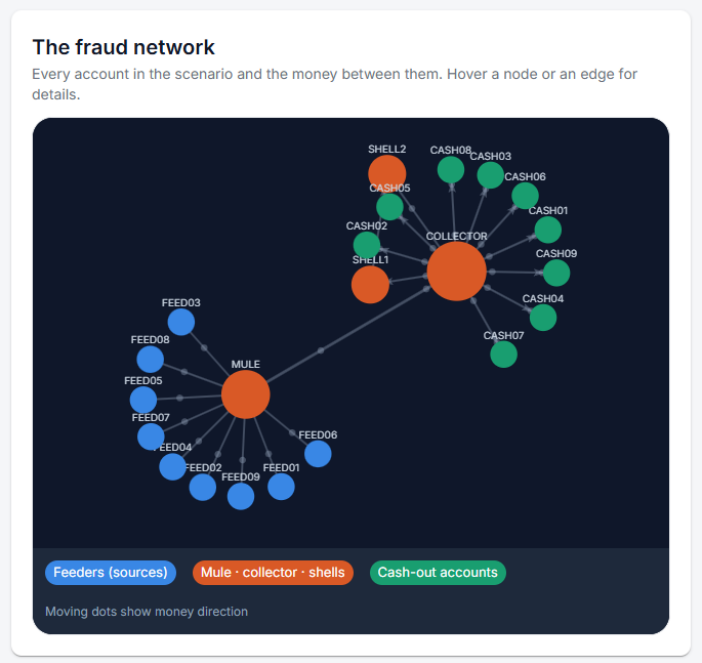


Figure 4.10: Demonstration Scenario — static screenshot of the interactive money-flow network for the scripted twenty-two-account scenario of Section 4.2.5, with the feeders, the mule and collector, the two shell accounts and the nine cash-out accounts coloured by role. Arrowheads at the receiving end of each edge mark the direction of money flow; in the running application dots also travel along the edges in that direction, and the small dots visible on the edges here are those markers captured at one instant

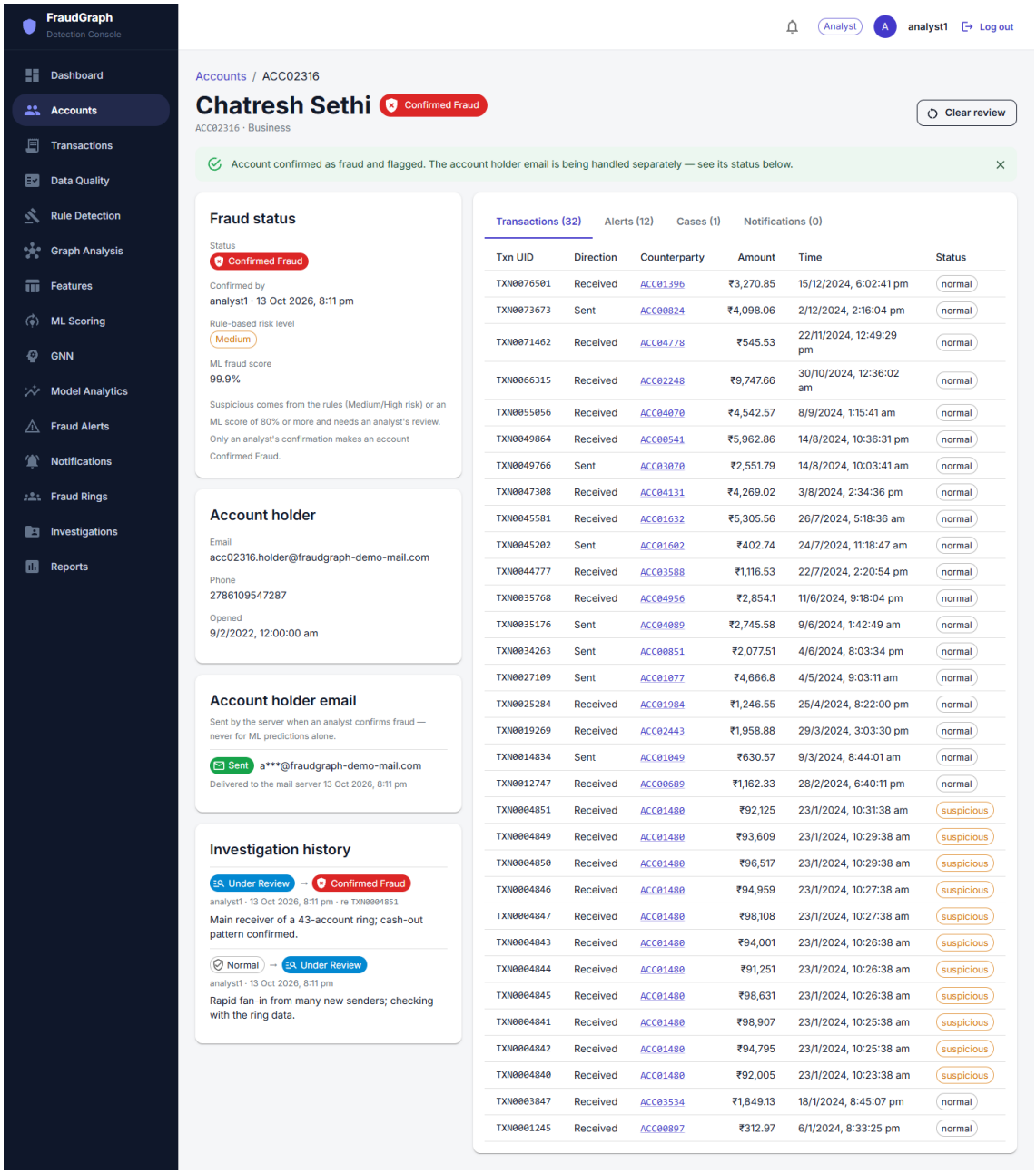


Figure 4.11: Account Investigation — a confirmed fraud account showing the analyst and timestamp that confirmed it, the rule-based risk level alongside the independent model score, the account-holder notification that was sent on confirmation with the recipient address masked, the audit trail of status transitions with the analyst's notes, and the account's suspicious transactions

Chapter 5

Coding

This chapter describes the key technologies used to implement Financial Fraud Detection Using Graph Machine Learning and presents representative source code from the most critical modules, including the rule engine, the graph analysis module, and the model training pipeline. The backend is entirely written in **Python**, with the analyst client in **JavaScript (React)**. The codebase is structured as a Python package under `backend/app/` with `requirements.txt` as the dependency manifest. This chapter introduces the primary languages and frameworks before presenting representative code snippets from the most critical modules. Every listing is taken from the application repository at commit `3800cc1`; where lines are left out, the omission is marked in the listing with a comment beginning `# ...`.

5.1 Python

Python 3.10 was chosen for its:

- Mature data and scientific ecosystem (pandas, NumPy, scikit-learn, XGBoost) which covers the entire feature engineering and modelling workload without leaving the language.
- First-class graph tooling through NetworkX, whose algorithm catalogue — PageRank, centrality measures, community detection and cycle enumeration — maps directly onto the structural patterns the project needs to detect.
- Strong typing via Pydantic v2 for request validation and response serialisation.
- Built-in `sqlite3`, `logging`, `datetime` and `bisect` standard library modules used extensively in the project.

Python's uniformity across the stack is a practical advantage here: the same language loads the dataset, runs the rules, builds the graph, trains the models and serves the API, so there is no serialisation boundary between the detection logic and the web layer.

5.2 FastAPI

FastAPI 0.111 provides the HTTP layer. Its key advantages in this project are:

- **Pydantic integration**: Request bodies and query parameters are automatically validated; invalid payloads return a structured 422 response without any hand-written checks.
- **Dependency injection**: Both the database session and JWT authentication are implemented as FastAPI dependencies and applied declaratively to each protected route, so a router simply declares `Depends(get_db)` in its signature.
- **Automatic OpenAPI documentation**: The interactive `/docs` interface is generated from the route signatures, which made the detection endpoints testable before the React client existed.
- **Router modularity**: Each resource lives in its own `APIRouter` module and is mounted under a common `/api` prefix, keeping the application factory small.

5.3 React

React 18, built with Vite 5, builds the analyst client. The application uses:

- **Material UI 5** for the component library and theming, giving the data-dense tables, dialogs and chips a consistent appearance without custom CSS.
- **Axios** with a shared instance that holds the base URL and an interceptor which attaches the bearer token and redirects to the login page on a 401 response.
- **React Router 6** for client-side routing, with a protected-route wrapper guarding every analytical page.
- **Recharts** for the dashboard visualisations — risk distribution, alert severity breakdown and transaction volume over time.

5.4 Sample Coding

5.4.1 rules.py — Rule Engine

Listing 5.1: `rules.py` — thresholds and the fan-in / fan-out rule (excerpt; omitted lines marked)

```
LARGE_AMOUNT = 100_000
RAPID_COUNT = 5
RAPID_WINDOW_MIN = 10
NEW_ACCOUNT_DAYS = 30
NEW_ACCOUNT_AMOUNT = 50_000
FAN_THRESHOLD = 8
FAN_WINDOW_H = 1
CIRCLE_WINDOW_H = 48

# ... RULE_LABELS, RULE_CODES and _load() omitted ...


def run_rules(txns: pd.DataFrame, accounts: pd.DataFrame) -> dict:
    """Return {rule_name: set(txn_uid)} for each rule."""
    hits = {name: set() for name in RULE_LABELS}
    if txns.empty:
        return hits
    txns = txns.sort_values("timestamp").reset_index(drop=True)

    # R1 large amount
    hits["large_amount"] = set(txns.loc[txns["amount"] > LARGE_AMOUNT, "txn_uid"])

    # ... R2 and R3 omitted ...

    # R4 fan-in / R5 fan-out (distinct counterparties within a window)
    fwin = timedelta(hours=FAN_WINDOW_H)

    def fan(group_col, counter_col, out_key):
        # Single pass per account: a Counter of the counterparties inside the
        # window gives the distinct count in O(1) per step, instead of slicing
        # the window and calling nunique() for every row.
        for _, g in txns.groupby(group_col):
            ts = g["timestamp"].tolist()
            others = g[counter_col].tolist()
            uids = g["txn_uid"].tolist()
            in_window = Counter()
            left = 0
            marked = 0  # uids[left:marked] are already in hits
            for right in range(len(ts)):
                in_window[others[right]] += 1
                while ts[right] - ts[left] > fwin:
                    in_window[others[left]] -= 1
                    if not in_window[others[left]]:
                        del in_window[others[left]]
                    left += 1
                if len(in_window) >= FAN_THRESHOLD:
                    hits[out_key].update(uids[max(left, marked):right + 1])
                    marked = right + 1

    fan("receiver_uid", "sender_uid", "fan_in")
    fan("sender_uid", "receiver_uid", "fan_out")

    # ... R6 omitted ...
    return hits
```

5.4.2 graph_analysis.py — Graph Construction and Metrics

Listing 5.2: `graph_build.py`, `graph_analysis.py` and `fraud_ring.py` — construction, metrics and ring assembly (abridged excerpts)

```
# --- graph_build.py ----------------------------------------------------------
def build_graph(db: Session) -> nx.DiGraph:
    G = nx.DiGraph()
```

```
    for uid, name, risk in db.query(Account.account_uid, Account.customer_name,
     Account.risk_level):
        G.add_node(uid, name=name, risk=risk)

    rows = db.query(
        Transaction.sender_uid, Transaction.receiver_uid, Transaction.amount,
    ).all()
    for sender, receiver, amount in rows:
        if G.has_edge(sender, receiver):
            e = G[sender][receiver]
            e["amount"] += amount
            e["count"] += 1
        else:
            G.add_edge(sender, receiver, amount=float(amount), count=1)
    return G


# --- graph_analysis.py -----------------------------------------------------
CYCLE_LENGTH_BOUND = 3


def analyze(db: Session) -> dict:
    G = build_graph(db)
    n, m = G.number_of_nodes(), G.number_of_edges()

    degree_centrality = nx.degree_centrality(G)
    pagerank = nx.pagerank(G, weight="amount")

    # ... weakly connected components omitted ...

    # Louvain communities (on the undirected projection)
    community_id, community_size = {}, {}
    communities = nx.community.louvain_communities(G.to_undirected(), seed=42)
    for i, com in enumerate(communities):
        for node in com:
            community_id[node] = i
            community_size[node] = len(com)

    # short directed cycles (money loops): 2-cycles and 3-cycles via set intersection.
    # (nx.simple_cycles is far too slow on ~80k edges; this is equivalent for length <= 3)
    succ = {u: set(G.successors(u)) for u in G.nodes}
    pred = {u: set(G.predecessors(u)) for u in G.nodes}
    cycle_nodes = set()
    for u, v in G.edges:
        if u < v and u in succ[v]:          # 2-cycle u <-> v (count once)
            cycle_nodes.update((u, v))
    for u, v in G.edges:
        common = succ[v] & pred[u]
        common.discard(u); common.discard(v)
        if common:
            cycle_nodes.add(u); cycle_nodes.add(v)
            cycle_nodes.update(common)

    # ... the cycle counters, money totals and the bulk insert into
    #     graph_metrics omitted ...


# --- fraud_ring.py ---------------------------------------------------------
def detect(db: Session) -> dict:
    # ... per-account scores, names and labels, and the rules fired on each
    #     transaction (rules_by_txn), loaded from the database, omitted ...

    # 1) suspicious-activity subgraph; each edge remembers which rules fired on it
    G = nx.DiGraph()
    sus_txns = db.query(
        Transaction.txn_uid, Transaction.sender_uid, Transaction.receiver_uid, Transaction.amount,
    ).filter(Transaction.status == "suspicious").all()
    for txn_uid, s, r, amt in sus_txns:
        _add_edge(G, s, r, amt)
        G[s][r]["rules"] |= rules_by_txn.get(txn_uid, set())

    # ... transactions between accounts whose ML score is at least
    #     ML_SCORE_FLOOR (0.7) are added to G the same way, omitted ...

    # 2) connected groups; split oversized components into communities so
```

```
    #    distinct rings inside one big suspicious blob separate out
    MAX_COMPONENT = 60
    groups = []
    for comp in nx.weakly_connected_components(G):
        if len(comp) < MIN_RING_SIZE:
            continue
        if len(comp) <= MAX_COMPONENT:
            groups.append(comp)
            continue
        sub_u = G.subgraph(comp).to_undirected()
        for com in nx.community.louvain_communities(sub_u, seed=42, resolution=2.0):
            if len(com) >= MIN_RING_SIZE:
                groups.append(set(com))

    rings = []
    for comp in groups:
        sub = G.subgraph(comp)
        received = {u: 0.0 for u in comp}
        sent = {u: 0.0 for u in comp}
        for u, v, d in sub.edges(data=True):
            sent[u] += d["amount"]
            received[v] += d["amount"]

        # 3) roles
        main = max(comp, key=lambda u: received[u])
        withdrawal = None
        outs = sorted(G.successors(main), key=lambda v: G[main][v]["amount"], reverse=True)
        for v in outs:
            if v in comp and sent.get(v, 0) <= received.get(v, 0):
                withdrawal = v
                break
        # ... every other member is given the role "source" ...

        member_scores = [scores.get(u, 0.0) or 0.0 for u in comp]
        avg_score = sum(member_scores) / len(member_scores)
        ring_rules = set().union(*(d["rules"] for _, _, d in sub.edges(data=True)))
        rule_score = min(len(ring_rules), RULES_FOR_FULL_SCORE) / RULES_FOR_FULL_SCORE
        risk_score = max(avg_score, rule_score)
        # ... the ring record (members, roles, flow, rules, risk level) and
        #     the write to rings.json omitted ...
```

In `fraud_ring.py`, `MIN_RING_SIZE` is 4 and `RULES_FOR_FULL_SCORE` is 3, so one distinct rule on a ring's own transactions gives a rule score of 0.33, two give 0.67 and three or more give 1.0.

5.4.3 baseline.py — Model Training and Scoring

The excerpt below is taken from `app/ml/baseline.py`; imports, the feature-importance summary and the write-back of scores to the database are omitted for space. Note that the three models handle class imbalance by three different mechanisms, and that Isolation Forest is unsupervised — it is fitted on the training features alone and never sees a label. Its `contamination` parameter, however, is set from the label array of the whole dataset, test partition included; Section 7.1.2 discusses that choice and measures its effect.

Listing 5.3: `baseline.py` — model training and scoring (abridged excerpt)

```
FEATURE_COLS = [
    "total_sent", "total_received", "txn_count_sent", "txn_count_received",
    "unique_receivers", "unique_senders", "avg_sent_amount", "avg_received_amount",
    "max_sent_amount", "max_received_amount", "txn_per_active_day",
    "in_degree", "out_degree", "degree_centrality", "pagerank", "in_cycle", "community_size",
]
```

```python
def train_and_score(db: Session, test_size: float = 0.25, seed: int = 42) -> dict:
    df = load_features()
    if df is None:
        raise ValueError("Features not built. Run feature engineering first.")

    X = df[FEATURE_COLS].values
    y = df["is_fraud"].values
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, stratify=y, random_state=seed,
    )
    n_pos, n_neg = int(y_train.sum()), int(len(y_train) - y_train.sum())

    results, models = {}, {}

    # --- Random Forest (class-weighted) ---
    rf = RandomForestClassifier(
        n_estimators=300, class_weight="balanced", random_state=seed, n_jobs=-1,
    )
    rf.fit(X_train, y_train)
    rf_score = rf.predict_proba(X_test)[:, 1]
    results["random_forest"] = _metrics(y_test, (rf_score >= 0.5).astype(int), rf_score)
    models["random_forest"] = rf

    # --- XGBoost (scale_pos_weight) ---
    xgb = XGBClassifier(
        n_estimators=300, max_depth=6, learning_rate=0.1,
        scale_pos_weight=n_neg / max(n_pos, 1),
        eval_metric="logloss", random_state=seed, n_jobs=-1,
    )
    xgb.fit(X_train, y_train)
    xgb_score = xgb.predict_proba(X_test)[:, 1]
    results["xgboost"] = _metrics(y_test, (xgb_score >= 0.5).astype(int), xgb_score)
    models["xgboost"] = xgb

    # --- Isolation Forest (unsupervised; contamination = observed fraud rate) ---
    iso = IsolationForest(
        n_estimators=200, contamination=float(y.mean()), random_state=seed, n_jobs=-1,
    )
    iso.fit(X_train)
    iso_pred = (iso.predict(X_test) == -1).astype(int)
    iso_score = -iso.score_samples(X_test)  # higher = more anomalous
    results["isolation_forest"] = _metrics(y_test, iso_pred, iso_score)

    # --- feature importance (from the best supervised model by F1) ---
    best_name = max(("random_forest", "xgboost"), key=lambda k: results[k]["f1"])
    best = models[best_name]
    # ... top-eight feature importances omitted ...

    # --- score ALL accounts with the best model, write back ---
    all_scores = best.predict_proba(X)[:, 1]
    # ... write-back to Account.fraud_score, the saved model and
    #     baseline_results.json omitted ...
```

5.4.4 gnn.py — Graph Neural Network Models

Both models share one two-layer architecture, differing only in the convolution class. The layers are used with PyTorch Geometric's defaults and are trained full-batch over the whole graph: no neighbour sampling is configured, and `SAGEConv` therefore aggregates a node's neighbours by their mean and adds a separately learned transformation of the node's own features. Neither layer learns a weight for an individual neighbour.

Listing 5.4: `gnn.py` — model definition and weighted training (excerpt; omitted lines marked)

```python
def _build_model(model_type: str, in_dim: int, hidden: int = 64, out_dim: int = 2):
    import torch
    from torch_geometric.nn import GCNConv, SAGEConv
```

```
    conv_cls = {"gcn": GCNConv, "graphsage": SAGEConv}[model_type]

    class Net(torch.nn.Module):
        def __init__(self):
            super().__init__()
            self.conv1 = conv_cls(in_dim, hidden)
            self.conv2 = conv_cls(hidden, out_dim)
            self.dropout = torch.nn.Dropout(0.3)

        def forward(self, x, edge_index):
            x = self.conv1(x, edge_index).relu()
            x = self.dropout(x)
            return self.conv2(x, edge_index)

    return Net()


def train(db: Session, source: str = "synthetic", model_type: str = "graphsage",
          epochs: int = 100, hidden: int = 64, lr: float = 0.01, seed: int = 42) -> dict:
    # ... data assembly and the train / validation / test masks omitted ...

    # class-imbalance handling: weighted cross-entropy
    y_train = data.y[train_mask]
    counts = torch.bincount(y_train, minlength=2).float()
    weights = counts.sum() / (2 * counts.clamp(min=1))

    model = _build_model(model_type, data.num_features, hidden=hidden)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr, weight_decay=5e-4)
    loss_fn = torch.nn.CrossEntropyLoss(weight=weights)

    # ... training-history bookkeeping omitted ...
    y_all = data.y
    for epoch in range(epochs):
        model.train()
        optimizer.zero_grad()
        out = model(data.x, data.edge_index)
        loss = loss_fn(out[train_mask], y_all[train_mask])
        loss.backward()
        optimizer.step()
        # ... periodic history recording omitted ...
```

The weights are the inverse class frequencies of the training partition. They are modest here rather than extreme: at the account level roughly a third of the seeded accounts carry a positive label, so the weighting corrects a moderate imbalance rather than a rare-event one. The severe imbalance in this dataset is at the transaction level, where only 3.33% of transactions are fraudulent — but the graph neural networks classify accounts, not transactions, so it is the account-level rate that applies to this loss.

Chapter 6

Testing and Validation

This chapter documents the testing strategy employed across the development lifecycle of Financial Fraud Detection Using Graph Machine Learning, covering unit tests, integration tests, alpha and beta testing phases, and the test cases the suite verifies. The project employs an automated test suite built with **pytest** and FastAPI's `TestClient`. Tests live in a `tests/` directory at the backend root and are run with `pytest` from there. Rather than mocking the application apart, most tests drive the real routers, dependencies, ORM session and services together against a temporary SQLite database created fresh for each test, so no run depends on another. The email tests deliver to a local `aiosmtpd` server, so SMTP behaviour is exercised rather than stubbed out.

6.1 Testing Methods

6.1.1 Unit Testing

Unit tests verify the behaviour of individual modules against a deterministic fixture dataset built in `conftest.py` — small enough that every expected outcome can be checked by hand.

Table 6.1: Unit Test Files and Coverage Areas

Test File	Coverage Area
`test_migrations.py`	Schema migration against an existing database and against a fresh one, with existing rows preserved
`test_accounts.py`	Risk level is written by the rule engine alone; the onboarding category is manual and affects nothing else
`test_fraud_review.py`	Status transition rules, confirmation, clearing, and the audit event each one writes
`test_model_analytics.py`	Reported model metrics recomputed from the stored test predictions

6.1.2 Integration Testing

Integration tests verify that multiple modules collaborate correctly across the API boundary:

Table 6.2: Integration Test Files

Test File	Coverage Area
`test_email.py`	Account-holder email: delivery, no duplicates, mail server down, SMTP unconfigured, missing address
`test_notifications.py`	Real-time push, persistence across sessions, retries, disconnect catch-up, stream authorisation, token expiry
`test_standalone_demo.py`	The scripted scenario run end to end through the real rule engine, graph analysis, alerts and ring detection
`test_review_fixes.py`	Regression tests, one or two per defect found in code review

6.1.3 Alpha Testing

Alpha testing was conducted internally by the development team. The following scenarios were verified:

- Uploading a transaction CSV and confirming that malformed rows are rejected with a readable error rather than being silently dropped.
- Running all six rules over the full dataset and checking that each generated alert carries the rule code and a human-readable reason.
- Running graph analysis and verifying that PageRank and community assignments are persisted for every account.
- Training the classical models and the graph neural networks, and confirming that the reported metrics are computed on a held-out split and not on the training data.
- Inspecting a detected fraud ring in the graph view and confirming that the member roles shown match the direction and size of the money flowing between them.
- Confirming an account as fraudulent and checking that the status event, the queued email, the notification and the alert all appear exactly once.
- Re-running a single pipeline stage and confirming that the other stages' results are preserved.

6.1.4 Beta Testing

Beta testing was informal. The system was deployed publicly — the FastAPI backend on Render's free tier and the React client on Vercel — and classmates were asked to use the analyst console without guidance beyond the login credentials. No structured questionnaire was administered and no formal usability metrics were collected, so this phase is reported as exploratory defect discovery rather than as a user study. Three defects reached the deployed build and were found this way:

1. **Cross-origin rejection.** The backend returned a correct 200 response to the login request, which the browser then discarded because the configured allowed origin did not match the client's. Corrected by normalising each configured origin to `scheme://host:port` before comparison.
2. **Rule-engine timeout.** The rule stage exceeded the client's request timeout on the full 80,000-transaction dataset. Corrected by rewriting the fan-in and fan-out rules as a single pass over a sliding window, reducing that stage from 26.9 to 4.6 seconds.
3. **Database loss between deployments.** The free-tier container's filesystem is ephemeral, so the database did not survive a redeploy. Corrected by re-seeding the bundled dataset at start-up when the database is found empty.

6.2 Test Cases

Table 6.3: Test Cases

TC#	Test Scenario	Input / action	Expected Output	Result
TC01	Schema migration	Start up against a pre-existing database	New tables and columns added; existing rows preserved	Pass
TC02	Fresh database	Start up with no database file	Full schema created from scratch	Pass
TC03	New account risk level	Create an account	Risk level starts at Low	Pass
TC04	Risk level is not editable	Attempt to set risk level through the account form	The submitted value is ignored; only the rule engine writes it	Pass
TC05	Onboarding category is separate	Set and then clear the onboarding risk category	Stored and cleared without affecting risk level, rules or scores	Pass
TC06	Rule engine sets risk	Run the rules over a crafted dataset	Risk level raised on the accounts the rules hit; onboarding category untouched	Pass
TC07	Analyst confirms fraud	Analyst confirms a suspicious account	Status becomes Confirmed Fraud; a status event records who, when and why	Pass
TC08	ML score is not a confirmation	A high model score is written to an account	The account stays Suspicious; only a human decision confirms	Pass
TC09	Status transition rules	Attempt each transition from each status	Only the permitted transitions are accepted	Pass
TC10	Authorisation on review	A non-analyst account calls the review endpoint	403 Forbidden	Pass

continued on the next page

Table 6.3 — continued from the previous page

TC#	Test Scenario	Input / action	Expected Output	Result
TC11	Account-holder email	Confirmation on an account with a valid address	Exactly one email queued and delivered to the local SMTP server	Pass
TC12	No duplicate email	The same confirmation submitted twice	Nothing changes; no second email	Pass
TC13	Mail server down	Confirm while the SMTP server is unreachable	The confirmation still stands; the failure is logged and can be retried	Pass
TC14	SMTP not configured	Confirm with no mail host set	Confirmation succeeds; the email is logged as not configured	Pass
TC15	No usable address	Confirm an account with no email address	Email skipped, not failed	Pass
TC16	Real-time alert	A confirmed-fraud account sends a new transaction	High-priority alert stored and pushed to every open notification stream	Pass
TC17	Only confirmed accounts notify	An unconfirmed suspicious account transacts	No notification raised	Pass
TC18	Notification persistence	Client logs out and back in	Stored notifications re-read from the database and redelivered	Pass
TC19	Disconnect catch-up	Client disconnects, then reconnects	Missed notifications delivered on reconnection	Pass
TC20	Stream authorisation	Open the notification stream without an analyst role	Stream refused	Pass
TC21	Token expiry closes stream	Hold a stream open until the token expires	Stream closed by the server	Pass

continued on the next page

Table 6.3 — continued from the previous page

TC#	Test Scenario	Input / action	Expected Output	Result
TC22	Model analytics correctness	Request analytics for a trained model	Reported metrics match those recomputed from the stored test predictions	Pass
TC23	Missing results reported	Request analytics with no stored results	The absence is reported rather than fabricated	Pass
TC24	Analytics authorisation	A non-analyst requests model analytics	403 Forbidden	Pass
TC25	GNN comparison on one split	Train GCN and GraphSAGE	Both evaluated on the same test split	Skipped
TC26	Demo scenario planting	Build the standalone demo database	Exactly the 22 scripted accounts and 22 transactions, and nothing else	Pass
TC27	Demo triggers every rule	Run the rule engine over the demo database	All six rules R1–R6 fire, each on its intended stage	Pass
TC28	Demo ring detection	Run graph analysis and ring detection on the demo	The mule network is recovered as a ring with roles assigned	Pass

6.3 Test Execution Results

Table 6.3 documents 28 test scenarios — the behaviours the suite is designed to establish, written for a reader. These are not in one-to-one correspondence with the automated test functions: a single scenario may be covered by several functions (and a few functions are parameterised, so one function contributes more than one executed test). The suite as executed collects 59 tests. The two counts describe the same body of work at different granularities and should not be added together.

The suite was executed in full against the current build with `pytest -q`. The outcome is summarised in Table 6.4. It was executed again during the final verification of this report, on a fresh checkout of commit `3800cc1` under Python 3.11.17 on Linux, and gave the same result — 57 passed and 2 skipped — with the same per-file counts.

Table 6.4: Test Suite Execution Summary

Test File	Covers	Tests	Result
`test_migrations.py`	Schema upgrade and fresh creation	2	2 passed
`test_accounts.py`	Risk level and onboarding category	8	8 passed
`test_fraud_review.py`	Confirmation, transitions, authorisation	5	5 passed
`test_email.py`	Holder email and its failure modes	8	8 passed
`test_notifications.py`	Real-time alerting and stream behaviour	7	7 passed
`test_model_analytics.py`	Reported metrics and authorisation	5	3 passed, 2 skipped
`test_review_fixes.py`	Code-review regressions	10	10 passed
`test_standalone_demo.py`	The scripted scenario end to end	14	14 passed
Total		**59**	**57 passed, 2 skipped**

No test failed. The two skipped tests both belong to the graph-neural-network comparison in `test_model_analytics.py`; they skip automatically when PyTorch Geometric is absent from the environment running the suite, so that the analytics for a model which was never evaluated are not reported as though they had been. Installing the optional GNN requirements enables them; they were not enabled in either run recorded here, so TC25 has not been exercised by the automated suite.

Chapter 7

Results and Discussion

This chapter evaluates Financial Fraud Detection Using Graph Machine Learning against the objectives set out in Chapter 1. It first describes the experimental setup, then reports the measured results, and finally identifies the advantages the design delivers and the limitations that constrain it.

7.1 Experimental Setup

7.1.1 Dataset

The primary dataset is synthetic. It was generated for this project with the Faker library and is loaded from `backend/data/raw/`. The seeded dataset is 5,000 accounts (`ACC00001`–`ACC05000`) and 80,000 transactions (`TXN0000001`–`TXN0080000`), of which 1,645 accounts and 2,663 transactions carry a positive fraud label. Fraud therefore accounts for 32.9% of accounts but only 3.33% of transactions — the account-level task is only moderately imbalanced, while the transaction-level task is severely so.

The working database contains three further accounts and four further transactions, created by hand through the user interface while the deployed instance was being tested and demonstrated, bringing its totals to 5,003 and 80,004. These records carry no meaningful ground-truth label — `is_fraud` is left at its default of 0 because nobody adjudicated them — so they are excluded from every evaluation reported in this chapter. Table 7.1 states the population each experiment actually used, and the final column names the stored output each figure was read from. Those output files are written locally by the application and are excluded from version control, so they cannot be inspected from the repository; Section 7.2.7 records which figures were regenerated from the committed code and data during the final verification.

All five therefore operate on the same seeded population. It is worth being precise about why, because the code does not enforce it: `build_graph()` queries every row of the `accounts` table with no filter, and `build_features()` iterates every account likewise.

Table 7.1: Dataset population used by each experiment

Experiment	Population	Evidence
Rule engine	80,000 seeded transactions	`transactions` rows with `txn_uid` $\leq$ `TXN0080000`
Feature engineering	5,000 seeded accounts	`features.csv`: 5,000 rows, last `account_uid` is `ACC05000`
Classical models	3,750 train / 1,250 test, 5,000 scored	`baseline_results.json`: `train_size` 3750, `test_size` 1250, `accounts_scored` 5000
Graph neural networks (synthetic)	3,188 train / 563 validation / 1,249 test	`gnn_results.json`: node counts per split, summing to 5,000
Graph analysis and ring detection	5,000 seeded accounts	`graph_metrics`: 5,000 rows; `rings.json` contains no account above `ACC05000`

Nothing in the pipeline excludes a manually created account. The stored artefacts contain 5,000 rows because they were produced from a database state that did not yet hold the three demonstration accounts — `features.csv` ends at `ACC05000`, `graph_metrics` holds no row for `ACC05001`–`ACC05003`, and `rings.json` names no account above `ACC05000`. **Re-running the pipeline against the working database today would include all 5,003 accounts and 80,004 transactions and would produce slightly different figures.** The results reported here are those of the stored run, and the scope column of Table 7.1 describes that run rather than a filter in the code. A database seeded from `data/raw/` alone contains exactly the seeded population, and the verification re-run of Section 7.2.7 reproduced every population in Table 7.1 from such a database.

The artefacts were not all written at the same moment. `features.csv`, `baseline_results.json` and `rings.json` share one timestamp and come from a single pipeline execution; `gnn_results.json` was written several days later from the same feature matrix, which is why the graph neural networks are evaluated on the same 5,000 accounts despite the later date. The database file itself has been modified more recently still, by the interface activity that created the demonstration records.

Ground truth comes from the generator, which plants fraud ring structures and records their members in the `is_fraud` column of the `accounts` and `transactions` tables. This column is written only when the dataset is loaded. The rule engine writes to `status` and `risk_level`, and the models write to `fraud_score`; neither ever modifies `is_fraud`. The supervised targets are therefore independent of the rule engine's predictions, and the reported model scores are not inflated by target leakage. A separate and weaker form of information sharing — the graph-derived features being computed over the whole network

before the split — is discussed in Section 7.1.2.

The second dataset is the public Elliptic Bitcoin dataset [8], loaded through PyTorch Geometric: 203,769 nodes, 234,355 edges and 165 anonymised node features, with the dataset's own temporal split.

7.1.2 Preprocessing, Splits and Hyperparameters

For the classical models, the seventeen features of Table 4.2 are used unscaled, and the data is divided by a stratified random split with `test_size = 0.25` and `random_state = 42`, giving 3,750 training and 1,250 test accounts (1,234 fraudulent accounts in the training partition).

For the graph neural networks the same seventeen features are standardised to zero mean and unit variance, and the split is produced by a separate routine that takes the first 25% of each shuffled class, with 15% of the remainder held out for validation. Flooring the per-class count yields 1,249 test nodes rather than 1,250. The classical models and the graph neural networks are therefore evaluated on two different, although identically-sized and identically-stratified, partitions of the same data. GCN and GraphSAGE do share one partition with each other, identified in the stored results by the split identifier `4f01af3d7815646e`, so the comparison between those two is exact. Recomputing the split from the feature matrix with the routine in `app/ml/gnn.py` reproduces both the node counts and that identifier.

The GNN evaluation on the synthetic graph is **transductive**: the whole graph, including the features and edges of the test nodes, is present during training and only the labels are masked. GraphSAGE is an inductive architecture, but it is not exercised inductively here.

The same caveat applies to the classical models, and it is easy to overlook. Six of the seventeen features — `in_degree`, `out_degree`, `degree_centrality`, `pagerank`, `in_cycle` and `community_size` — are computed by `graph_analysis.analyze()` over the complete account network, before any split is made. PageRank and Louvain community membership are global quantities: a test account's PageRank depends on the edges of training accounts, and a training account's PageRank depends on the edges of test accounts. The split that follows holds out labels, not accounts.

This is not label leakage — no component writes `is_fraud`, and the structural features are computed from the transaction graph alone, which carries no fraud annotation. But it does mean the classical evaluation measures something narrower than it might appear to. What is being measured is the ability to classify accounts whose labels were held out, with the full graph context of those accounts available during feature construction. It is not a measurement of how the models would perform on accounts that were entirely absent when the graph was built — a new customer, say, whose edges had never been seen.

That experiment would require rebuilding the graph and recomputing the features on the training accounts alone, and it was not performed. No claim of inductive generalisation to unseen accounts is made anywhere in this report.

Hyperparameters are given in Table 7.2. All runs use seed 42.

Table 7.2: Model Configuration

Model	**Type**	**Configuration**
Random Forest	Supervised	300 trees; `class_weight="balanced"`
XGBoost	Supervised	300 estimators; max depth 6; learning rate 0.1; `scale_pos_weight` $= n_{\text{neg}}/n_{\text{pos}}$; `eval_metric="logloss"`
Isolation Forest	Unsupervised	200 estimators; `contamination = y.mean()` $= 0.3290$, the fraud rate of the full label array, as committed (a training-only estimate, 0.3291, gives identical predictions; see below). Fitted on features only — it is never given a label
GCN	Graph, transductive	2 `GCNConv` layers; hidden 64; dropout 0.3; Adam, lr 0.01, weight decay 5×10^{-4}; 100 epochs; weighted cross-entropy
GraphSAGE	Graph	2 `SAGEConv` layers; otherwise as GCN

Note that the imbalance mechanisms are distinct and are not interchangeable: Random Forest reweights the Gini criterion, XGBoost rescales the positive class gradient, the graph neural networks weight the cross-entropy loss, and Isolation Forest does none of these — it is an unsupervised detector whose contamination parameter merely states the expected proportion of anomalies.

That parameter nevertheless deserves care. `fit()` receives no labels, so the model is unsupervised in the strict sense; but `contamination` decides where the decision threshold falls. In the evaluated code — every commit of the application repository up to and including `3800cc1` — it is set to `float(y.mean())`, the fraud rate of the whole label array, test partition included (Listing 5.3). That lets the test labels influence the evaluation configuration, which is a methodological flaw even though no label reaches `fit()`. The sound choice is the training-partition estimate `float(y_train.mean())`. That change has not been committed to the application repository. Its effect was instead measured directly during the final verification: on the same feature matrix, split and seed, the two settings (0.329000 against 0.329067, a difference of 6.7×10^{-5}) produce identical test predictions — the same confusion matrix, 654 true negatives, 185 false positives, 182 false negatives and 229 true positives — and therefore identical metrics, under both library sets of Table 7.5. The flaw is real in principle but changes no figure in this chapter, and the Isolation

Forest row of Table 7.3 stands for either setting.

The original measurements were taken on a CPU-only Windows laptop; no GPU was used. The verification re-runs used a CPU-only Linux container (Table 7.5).

7.2 Results

7.2.1 Rule Engine

Evaluated against the transaction-level ground truth over the 80,000 seeded transactions, the rule engine flagged 2,565, of which 2,562 were genuinely fraudulent:

	True positives	False positives	False negatives	Recall
Rule engine R1–R6	2,562	3	101	96.21%

Precision is 99.88% and recall 96.21%. Across the whole working database the engine flags 2,567 transactions and produces 2,981 individual rule hits, since a transaction may violate more than one rule: R2 (rapid transactions) 1,567, R4 (fan-in) 642, R5 (fan-out) 413, R6 (circular) 168, R3 (new-account large credit) 131 and R1 (large amount) 60. Two of those 2,567 are the manually created demonstration transactions excluded by Table 7.1; counting them as false positives, as their default `is_fraud = 0` would require, would give 5 false positives and a precision of 99.81%. Either figure is defensible — what is not defensible is quoting one and computing it from the other, so the labelled seeded population is used throughout.

Re-running the rule engine over the seeded data alone at commit `3800cc1` reproduced the 2,565 / 2,562 / 3 / 101 counts exactly. On that population the engine produces 2,979 rule hits, with every per-rule count as above except R3, at 129; the two additional R3 hits in the working-database figures belong to the two manually created transactions.

The high precision is expected rather than impressive: the generator plants fraud that matches these typologies, so the rules are being measured against patterns designed to match them.

7.2.2 Model Performance

Table 7.3 reports every metric recorded by the evaluation runs. The values are taken from the two result files the application writes, `baseline_results.json` and `gnn_results.json`. Figure 7.1 presents the same figures graphically.

The Random Forest and Isolation Forest rows were reproduced exactly by the verification re-run with scikit-learn 1.5.2. The XGBoost row was not: re-running at `3800cc1` gives accuracy 0.889, precision 0.832, recall 0.830, F1 0.831 and ROC-AUC 0.944, identically

Table 7.3: Model Performance on the Synthetic Account Graph

Model	Acc.	Prec.	Recall	F1	ROC-AUC	PR-AUC	Test n
Random Forest	0.888	0.856	0.793	0.823	0.946	—	1,250
XGBoost	0.886	0.828	0.822	**0.825**	0.945	—	1,250
Isolation Forest	0.706	0.553	0.557	0.555	0.759	—	1,250
GCN	0.696	0.527	0.737	0.615	0.776	0.681	1,249
GraphSAGE	0.891	0.816	0.864	**0.839**	0.957	0.928	1,249

under every XGBoost release tried from 2.0.3 to 3.2.0. The stored figures are retained because they are the recorded result of the evaluation run, but they should be read as reproducible to within about 0.006 in F1 rather than exactly; the cause of the difference could not be established from the repository, which records neither the XGBoost build nor the platform of the original run. Under the latest scikit-learn the Random Forest result also shifts (F1 0.824, with precision 0.816 and recall 0.832), so the precision–recall balance of that model depends on the library version even where its F1 does not.

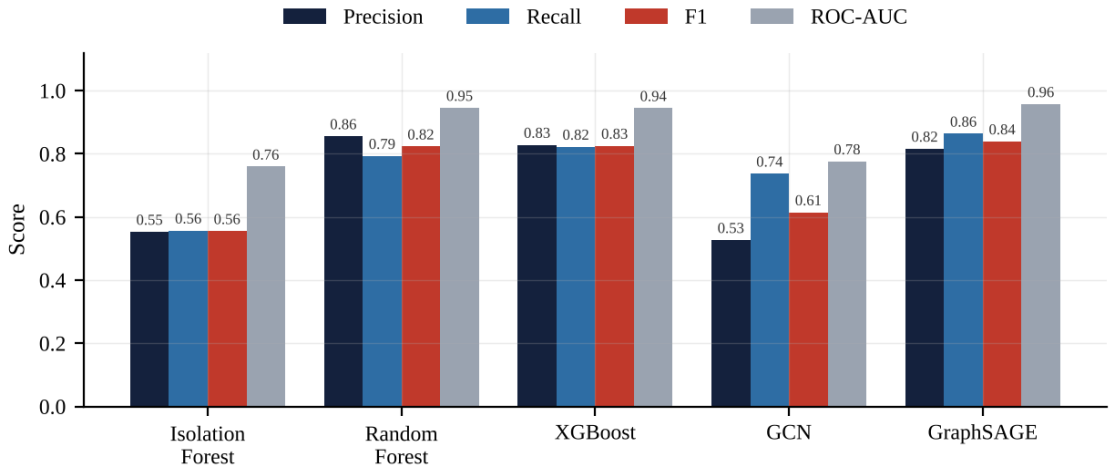


Figure 7.1: Model performance on the synthetic account graph. Classical models (left three) and graph neural networks (right two) are evaluated on different partitions of the same data.

GraphSAGE records the highest F1 at 0.839, ahead of XGBoost at 0.825. That margin — 1.4 points, or 0.8 points against the re-run XGBoost figure — is small, and the two models were not evaluated on the same partition, so it should be read as an indication rather than as a measured improvement. Establishing that the difference is real would require evaluating both on identical folds, repeated across several seeds, which was not done. What the results do show clearly is the gap between GCN (F1 0.615) and GraphSAGE (F1 0.839) on an identical split: a substantial difference. The two architectures do differ in how they combine a node with its neighbourhood — the graph convolution of Kipf and

Welling [6] folds a node into a single degree-normalised sum over itself and its neighbours, whereas the `SAGEConv` layer used here aggregates the neighbours by their mean and adds a separately learned transformation of the node's own features [7]. Neither layer learns an individual weight for each neighbour; that is the mechanism of attention-based and selection-based models such as CARE-GNN [9], which is not used here. This experiment establishes that GraphSAGE scored higher on the shared partition; it does not establish which architectural difference caused the gap, and an ablation isolating that mechanism was not run.

Isolation Forest is much the weakest at F1 0.555, which is the expected result for an unsupervised detector that is given no labels and must treat fraud as generic anomaly.

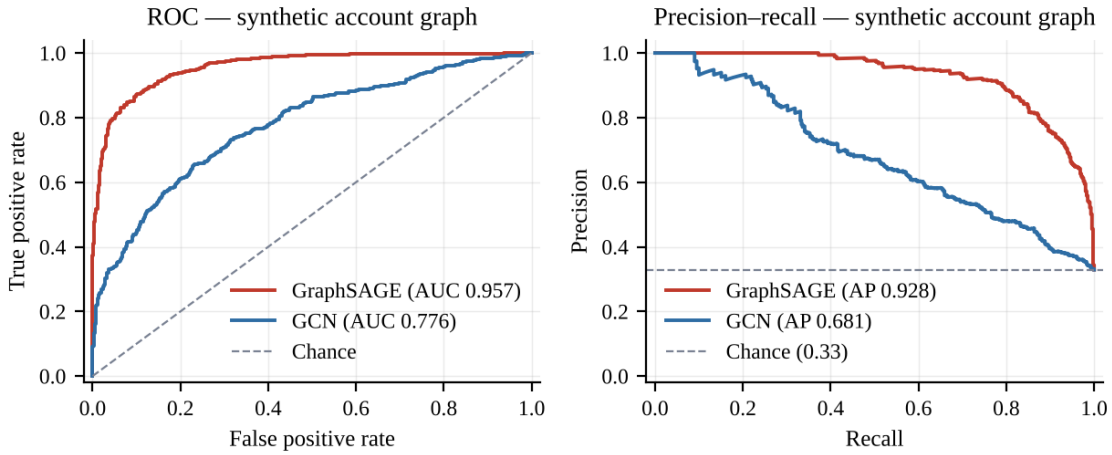


Figure 7.2: ROC and precision–recall curves for the two graph neural networks on the synthetic account graph, computed from the stored per-node test predictions.

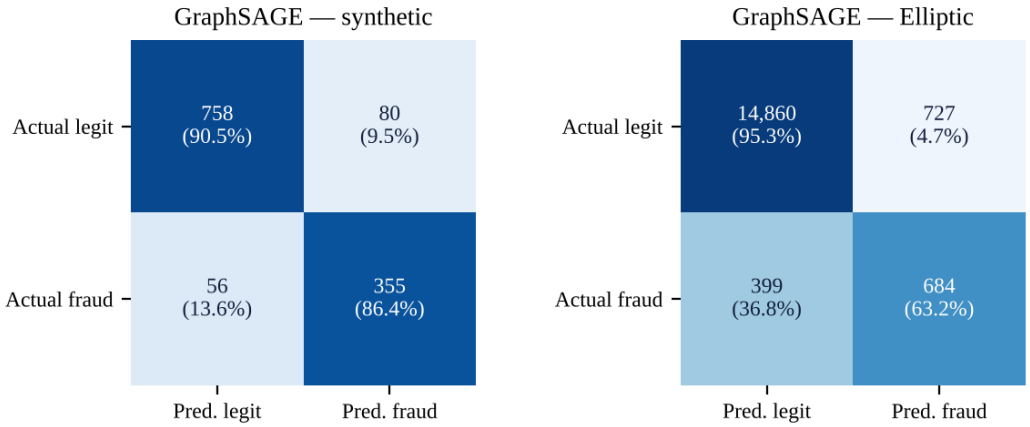


Figure 7.3: Confusion matrices for GraphSAGE at the 0.5 decision threshold. Left: the synthetic account graph. Right: the Elliptic Bitcoin dataset.

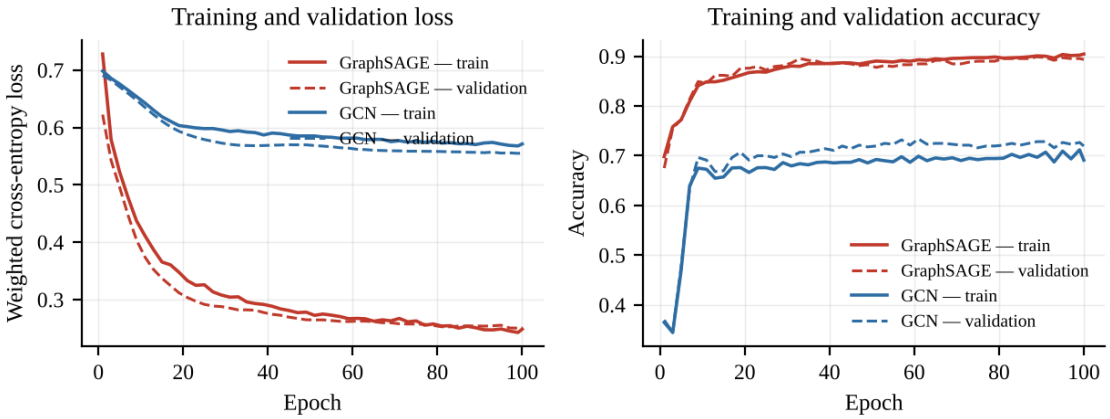


Figure 7.4: Training and validation curves over 100 epochs. Neither model shows a widening train–validation gap, so overfitting is not evident at this depth.

7.2.3 Feature Importance

Figure 7.5 shows the eight highest-gain features of the trained XGBoost model. The ranking is dominated by transaction-level magnitude features: `max_sent_amount` alone accounts for 0.296 of total gain and `max_received_amount` for a further 0.199. Only one graph-derived feature, `pagerank`, appears in the top eight, at 0.056. The graph features are therefore a minor contributor to the classical model, and an earlier claim that they rank consistently among the most important is not supported by the measured importances. The verification re-run, whose XGBoost metrics differ slightly as noted above, gives the same conclusion: the same eight features, `max_sent_amount` (0.280) and `max_received_amount` (0.206) first and second, and `pagerank` (0.055) again the only graph-derived feature, in fourth place.

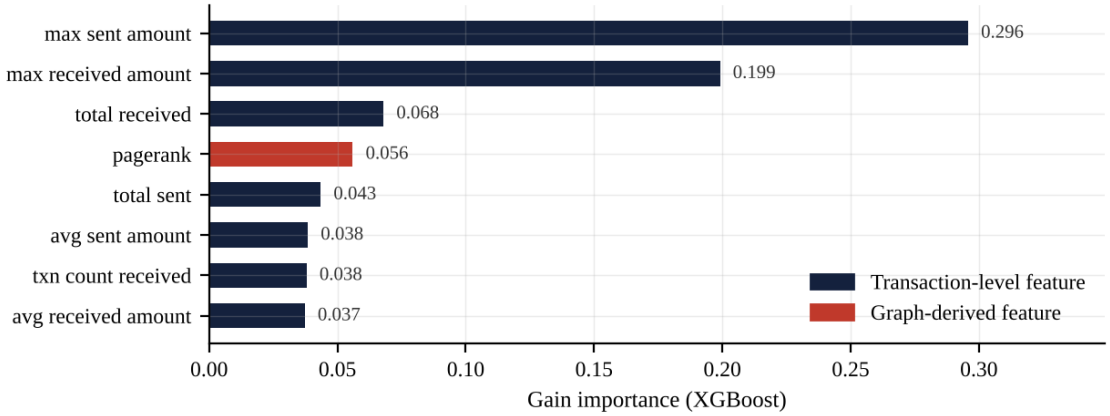


Figure 7.5: XGBoost gain importance, eight highest-ranked features.

7.2.4 Fraud Ring Detection

Ring detection assembled 52 rings covering 1,699 accounts, with a combined flow of Rs. 12.65 crore. Of the 1,699 member accounts, 1,644 carry a positive ground-truth fraud label, giving a purity of 96.8%. Ring sizes range from 17 to 54 members with a median of 31, and all 52 were rated High risk by the combined model-score and rule-evidence measure described in Section 4.2.4 and Listing 5.2. The verification re-run reproduced each of these counts exactly.

7.2.5 Elliptic Bitcoin Benchmark

Table 7.4: Graph Neural Network Results on the Elliptic Bitcoin Dataset

Model	Acc.	Prec.	Recall	F1	ROC-AUC	PR-AUC
GCN	0.920	0.415	0.560	0.477	0.841	0.480
GraphSAGE	0.933	0.485	0.632	0.549	0.898	0.606

On 16,670 test nodes, of which 1,083 are illicit, GraphSAGE reaches F1 0.549. Accuracy of 93.3% is uninformative here because 93.5% of the test nodes are legitimate; precision, recall and PR-AUC are the meaningful measures, and PR-AUC of 0.606 against a base rate of 0.065 shows the model is finding real signal.

The difference between F1 0.839 on the synthetic graph and F1 0.549 on Elliptic should *not* be read as a measurement of how far this system's accuracy would fall on real data. The two experiments differ in almost every respect: the nodes are accounts in one case and individual Bitcoin transactions in the other; the features are seventeen interpretable engineered values versus 165 anonymised ones; the positive rate differs by a factor of five; and the splits are random-stratified versus temporal, the latter being considerably harder because the model must generalise forward in time across a documented market shock. The Elliptic figures are best read as an independent check that the implementation works on real data at all, not as a transfer measurement.

7.2.6 Runtime

The application's stage display on the development laptop showed a single run of the detection pipeline — rule evaluation, graph construction and analysis, feature engineering, classical model training and scoring, alert generation and ring assembly — over the full transaction set completing in approximately 22 seconds. **This figure is an informal observation, not a benchmark.** It was read once from the screen; the pipeline does not persist its timings, so no artefact records it; and it was not possible to establish from the

repository whether it was measured before or after the demonstration records were added. It is not a guaranteed or typical runtime, and it cannot be independently reproduced.

During the final verification the same stages, preceded by the cleaning step, were timed with a wall-clock timer on the Linux container described in Table 7.5. Two single runs took 16.9 and 18.7 seconds in total — rule engine 6.3 and 6.9 s, graph analysis 5.0 and 5.8 s, model training 2.7 and 2.5 s, and every other stage under 1.5 s — and the per-stage timings are stored with the verification record. These are two runs on different hardware from the laptop, not a controlled benchmark; they neither confirm nor contradict the 22-second observation, and show only that a full run over the seeded data takes seconds rather than minutes, as the performance requirements of Section 3.4.1 ask.

Graph neural network training is *not* included: it is a separate, manually triggered step on the optional branch of Figure 4.5. Its timings, unlike the pipeline's, are persisted — `gnn_results.json` records 19.5 seconds for GraphSAGE and 11.9 seconds for GCN on the synthetic graph, and 99.1 seconds for GraphSAGE on Elliptic — although that file, like the other result files, is held on the development machine rather than in the repository.

An earlier optimisation is worth recording. The fan-in and fan-out rules originally re-sliced the time window for every row and recomputed the distinct-counterparty count, which dominated the run. Rewriting them to maintain a counter across a sliding window reduced the rule stage from 26.9 to 4.6 seconds on the development laptop, with outputs verified identical on the seeded dataset and on thirty randomised dense-window trials.

7.2.7 Reproducibility

The results in this chapter were produced by the application in the repository below, but not every figure can be regenerated from the repository alone. This section records the provenance of the evaluation, the outcome of a verification re-run made while this report was being finalised, and the evidential status of each result.

Building the dataset and running the pipeline. From `backend/` with the virtual environment active:

Listing 7.1: Reproducing the evaluation

```
python -m scripts.init_db         # create the tables and the administrator user
python -m scripts.seed_data       # load data/raw/: 5,000 accounts, 80,000 transactions
uvicorn app.main:app --reload     # API on 127.0.0.1:8000, interactive docs at /docs
```

The detection stages are then run in order, either from the analyst interface or by posting to the API: `/api/preprocessing/clean`, `/api/rules/run`, `/api/graph/analyze`, `/api/features/build`, `/api/ml/train`, `/api/alerts/generate` and `/api/rings/detect`. The graph neural networks are trained separately through `/api/gnn/train` and are not part of that sequence.

Table 7.5: Evaluation Provenance and Reproduction Environment

Item	Value
Repository	`https://github.com/Aarjav-333/fraud-detection-graphml`
Commit verified	`3800cc1` (14 October 2026), the head of `main` when this report was finalised; the repository carries no release tag. Every re-run in this section was made from this commit.
Original evaluation run	Commit not recorded: the result files store no commit identifier. The ring risk scores use the rule-evidence scoring introduced in `892ea22` (8 October 2026) and the GNN results use the evaluation format introduced in `9f0f5e9` (13 October 2026), so the stored results were produced at or after those commits. The model configuration in `baseline.py` is unchanged since the initial commit `d101f2c`; later changes to the evaluation modules concern only file handling.
Isolation Forest `contamination`	`float(y.mean())` at every commit, including `3800cc1`. The training-only estimate `float(y_train.mean())` is not in the repository; it was evaluated separately during the verification (Section 7.1.2).
Original environment	Python 3.11.9 in a virtual environment created from `backend/requirements.txt`. Package versions were not recorded, and the requirements files use >= constraints.
Key packages	FastAPI, SQLAlchemy 2, pandas, NetworkX, SciPy, scikit-learn, XGBoost, Faker; the graph neural networks additionally need `backend/requirements-gnn.txt` (PyTorch and PyTorch Geometric, roughly 200 MB)
Hardware	Original runs: CPU-only Windows laptop; no GPU was used at any stage
Verification environment	Python 3.11.17 on a Linux container with 4 vCPUs; no GPU. Run A: scikit-learn 1.5.2, XGBoost 2.1.4, pandas 2.2.3, NumPy 2.0.2, SciPy 1.14.1, NetworkX 3.4.2. Run B: the then-current releases (scikit-learn 1.9.1, XGBoost 3.2.0, pandas 3.0.6, NumPy 2.4.6, SciPy 1.17.1, NetworkX 3.6.1). PyTorch could not be installed, as its package host was not reachable from the container.
Deployed client	`https://fraud-detection-graphml.vercel.app`
Deployed API	`https://fraud-detection-backend.onrender.com`
Random seed	42 throughout — the train/test split, all three classical models, the Louvain community detection and both graph neural networks

Running the tests. `python -m pytest` from `backend/`. The suite creates its own temporary database and does not touch the working one.

The verification re-run. A fresh clone of `3800cc1` was seeded from `data/raw/` into an empty database, and the stages above were called in the same order through the application's own modules, once with each library set of Table 7.5. The Isolation Forest comparison and the recomputation of the GNN split were run on the feature matrix that the re-run produced. The scripts and their complete outputs are kept in the `verification/` directory of the report repository. Table 7.6 classifies every result in this chapter by the evidence behind it.

Table 7.6: Evidential Status of the Chapter 7 Results

Status	Results	Basis
Reproduced exactly from the committed code and data	Dataset counts and labels; rule-engine counts, precision and recall, and the per-rule hits on the seeded data; the classical split (3,750 / 1,250, 1,234 fraudulent in training); the Random Forest and Isolation Forest rows of Table 7.3; all fraud-ring figures; the GNN split sizes and split identifier	Verification re-run A at `3800cc1`
Reproduced approximately	The XGBoost row of Table 7.3 (re-run F1 0.831 against 0.825) and the importances of Figure 7.5; the Random Forest row under the latest scikit-learn	Re-runs A and B
Supported by stored artefacts only	GCN and GraphSAGE results on both datasets, Figures 7.2–7.4 and the GNN parts of Figure 7.1, GNN training times	Result files on the development machine; not in version control and not regenerated. The confusion matrices were checked against the reported metrics
Snapshot of the working database	Row counts in Table 4.1; the 2,567-flag, 2,981-hit and 99.81% figures of Section 7.2.1	Depend on manually created records and on alert-generation history
Informal observation	The 22-second pipeline runtime	No artefact; read once from the screen

Where the results are written. Table 7.7 lists the files the figures and tables in this chapter were read from. All paths are relative to `backend/`. Every one of them is listed in the application's `.gitignore`: they are regenerated on each run and are not part of the

repository, so the copies from which this chapter was written exist only on the development machine.

Table 7.7: Result Artefacts

File	Contents
`data/processed/features.csv`	The 17-feature matrix, one row per account
`app/ml/saved_models/baseline_results.json`	Split sizes, class balance, the metrics of all three classical models, and the feature importances
`app/ml/saved_models/rings.json`	Every detected ring with its members, roles, money flow and ground-truth count
`app/ml/saved_models/gnn_results.json`	Per-run GNN metrics, training curves, split identifiers and training times
`app/ml/saved_models/*_test_predictions.npz`	The raw test-set labels and predicted probabilities, from which the ROC, precision–recall and confusion-matrix figures of Section 7.2.2 are drawn
`app/ml/saved_models/xgboost.joblib`	The trained best-performing classical model

What the deployment can and cannot do. The hosted instance runs the rule engine, the graph analysis, feature engineering, the classical models, alert generation, ring detection and the full investigation workflow. It does *not* carry PyTorch or PyTorch Geometric, so the graph neural network results in this chapter — both the synthetic and the Elliptic runs — were produced locally and cannot be regenerated on the deployed instance. The `/api/system/info` endpoint reports this, and the interface hides the training controls accordingly.

7.3 Advantages and Limitations

7.3.1 Advantages

1. **Relational Detection**: Because transactions are modelled as a graph rather than a flat table, patterns that are invisible at the level of an individual transaction — fan-in collection, fan-out dispersal and closed-loop layering — become measurable structural properties. This is the central capability that transaction-level systems lack.
2. **Layered Defence**: Five complementary techniques operate over the same data. The rule engine catches the explicit, policy-defined cases; graph analysis catches the structural cases; the supervised classifiers catch combinations of signals that no single rule

encodes; Isolation Forest catches accounts that are simply anomalous without resembling any labelled example; and the graph neural networks learn structure the hand-chosen features do not express. An account that evades one layer is still exposed by the next.

3. **Graduated Explainability**: Explanations differ in kind by layer, and the system does not claim that every output is interpretable. Rule alerts carry an explicit rule code and a generated description, and are fully explainable. The classical models are accompanied by global feature importances (Figure 7.5), which explain the model's behaviour overall but not any individual decision; the system implements no per-prediction attribution such as SHAP or LIME. A model alert therefore states a probability and the interpretable feature values behind it, which is weaker than a rule explanation. The graph neural networks produce a score with no accompanying explanation at all, and their output is best treated as a ranking aid for an analyst rather than as a justified decision.

4. **Complete Investigation Workflow**: Detection is carried through to a decision. Alerts are grouped into rings with member roles assigned, rings are escalated into tracked cases with an assignee and a resolution, and cases are exported as PDF reports. The system is usable as an analyst's working tool rather than only as a detector.

5. **Stage Independence**: The four detection stages are separate endpoints that each persist their own output. An analyst can re-run model scoring after adjusting a threshold without recomputing the graph, which makes the system practical to iterate on.

6. **Portable Persistence**: All database access goes through the SQLAlchemy ORM and no module writes raw SQL, so the model classes and queries are expressed once and are largely independent of the engine beneath them. This narrows a migration to SQLite's permissive typing. It does not make one free: moving to PostgreSQL would still require installing and configuring a driver, creating the schema and migrating the data, adding the foreign-key constraints the current schema leaves to application code (Section 7.3.2), checking query and transaction behaviour on the new engine, and re-running the test suite against it. The ORM reduces the work; it does not reduce it to a connection string.

7. **Deployed and Re-runnable**: The system runs as a public deployment with the backend and frontend hosted separately, environment-driven configuration, and an automatic dataset seed on first boot, so a reviewer can reach a working instance without a local setup. The seed data is committed, so the rule-engine, classical-model and ring results can be regenerated locally from the repository (Section 7.2.7).

7.3.2 Limitations

1. **Synthetic Evaluation Data**: The dataset is generated, not real. Its fraud structures were planted by the generator, which means the models are being asked to recover patterns that were put there deliberately. The reported metrics therefore measure internal consistency, not real-world detection performance, and no claim is made about transfer to live banking data.
2. **Batch Rather Than Streaming**: The pipeline runs over a stored dataset. There is no incremental path that scores a single transaction as it arrives, so the system cannot block a payment in flight; it can only report on history already recorded.
3. **Rule Thresholds Are Calibrated, Not Regulatory**: The numeric thresholds used by the six rules were tuned to produce a workable alert volume on this dataset. They are modelled on standard anti-money-laundering typologies but they are not the statutory reporting figures of any jurisdiction, and they would have to be reset against local regulation before any real use.
4. **Graph Recomputation Cost**: Graph metrics are recomputed over the whole graph rather than updated incrementally. On the eighty-thousand-transaction dataset this is acceptable, but the cost of PageRank, community detection and cycle enumeration grows with the graph, so the current approach would not scale to a dataset orders of magnitude larger without incremental algorithms.
5. **Uniform Neighbour Aggregation**: Both graph neural networks weight every neighbour of an account alike. A fraudster who deliberately transacts with many legitimate accounts can therefore dilute their own signal — the camouflage attack that selection-based detectors such as CARE-GNN were designed to resist. No such defence is implemented here.
6. **A Large Optional Dependency**: The graph neural networks require PyTorch and PyTorch Geometric, a download of several hundred megabytes that the free-tier deployment does not carry. The application degrades gracefully when they are absent, but the GNN results in this report were produced locally rather than on the deployed instance.
7. **Held-Out Labels, Not Held-Out Accounts**: Every evaluation in this report holds out labels while leaving the account present in the graph from which the features were computed. The graph neural networks are transductive by construction, and the classical models inherit the same property through the six global graph features. Nothing here measures performance on an account that was absent when the network was built, which is the situation a deployed system faces whenever a new customer appears.

8. **Relational Integrity Is Not Enforced by the Database**: Tables are joined on business keys — `account_uid`, `txn_uid`, `alert_uid` — rather than on declared foreign keys, the one exception being `notification_reads`. Referential consistency therefore rests on application code rather than on the database, which is tolerable for a single-process SQLite deployment but would have to be tightened before the schema was moved to a shared server.

9. **Single-Tenant Design**: The data model has no notion of an institution or a tenant. All accounts and transactions share one namespace, which is adequate for a single analyst team but prevents deployment as a shared service.

10. **Ring Assembly Is Heuristic**: Candidate rings are formed from connected components of the suspicious subgraph, and roles are assigned from the money flow — the largest receiver becomes the main account and its largest onward recipient that sends no more than it receives becomes the withdrawal account. This is a reasonable first approximation but it has no notion of confidence, so a large component may be reported as one ring where an investigator would separate it into several.

11. **Evaluation Artefacts and Environment Not Archived**: The result files are excluded from version control, the dependency versions are not pinned, and the Isolation Forest contamination estimate in the committed code still uses the full label array. As a result the graph neural network figures cannot be regenerated from the repository alone, and the XGBoost figures were reproduced only approximately (Section 7.2.7). Archiving the result files with the commit that produced them, recording the installed package versions, and committing the training-only contamination estimate would close these gaps.

Chapter 8

Conclusion and Future Scope

Financial Fraud Detection Using Graph Machine Learning demonstrates that treating a transaction history as a network rather than as a list of independent events changes what can be detected. The system successfully achieves the objectives set at the outset. Transactions are modelled as a directed weighted graph in which accounts are nodes, and NetworkX computes PageRank, degree centrality, Louvain community membership and cycle membership for every account. A transparent rule engine applies six checks derived from standard anti-money-laundering typologies, each alert carrying the rule code and a readable reason. A feature engineering module combines eleven transaction-level aggregates with six graph-derived metrics into a seventeen-feature vector per account, and Random Forest, XGBoost and Isolation Forest models score every account from that matrix. Graph Convolutional Network and GraphSAGE models learn from the account graph directly alongside them. The imbalance of the fraudulent class is addressed by a different mechanism in each family — balanced class weights for Random Forest, a positive-class weight for XGBoost, a contamination rate for the unsupervised Isolation Forest, and a weighted cross-entropy loss for the two graph neural networks. Detection output is carried through to a complete investigation workflow in which alerts are grouped into fraud rings with member roles assigned, an analyst confirms or clears each account against an audit trail, the holder of a confirmed account is notified by email, a further transaction sent by a confirmed account pushes a real-time alert to the analyst — a notification, not a block on the payment — and cases are exported as PDF reports. A React single-page application provides the analyst interface over a FastAPI service and a SQLite database accessed entirely through the SQLAlchemy ORM.

The system comprises a FastAPI backend of eighteen routers and seven detection modules, a React client of eighteen pages, an automated test suite of fifty-nine tests across unit, integration and workflow layers, and a public deployment with the backend and frontend hosted separately. Evaluated on the seeded synthetic dataset, the rule engine reached 96.21% recall at 99.88% precision, GraphSAGE reached an F1 score of 0.839 against XGBoost's 0.825, and 52 fraud rings were assembled with 96.8% of their members genuinely

fraudulent. On the independent Elliptic Bitcoin benchmark GraphSAGE reached an F1 score of 0.549. These figures are drawn from the evaluation outputs the application itself writes, and the caveats attached to them in Chapter 7 — a synthetic primary dataset, two different test partitions, and evaluations that hold out labels rather than accounts, so that no result here speaks to performance on an account the graph had never seen — apply equally here. A verification re-run from the committed code and data reproduced the rule-engine, Random Forest, Isolation Forest and fraud-ring results exactly and the XGBoost result to within 0.006 in F1 (0.831 on re-run); the graph neural network figures rest on stored result files that could not be regenerated in that verification. The roughly twenty-two-second pipeline runtime is an informal observation from the development laptop rather than a benchmark; the verification re-run, on different hardware, completed in 17 to 19 seconds.

In terms of future enhancements, the most immediate priority is evaluation on a real labelled transaction graph rather than a synthetic one. The gap between the synthetic and the Elliptic results reported in Chapter 7 shows that performance on planted structures says little about performance in the field, and only real data settles it. Closely related is defending the graph neural networks against camouflage: both currently weight every neighbour alike, so a fraudster who transacts widely with legitimate accounts dilutes their own signal, and a neighbour selection mechanism of the kind CARE-GNN introduced would address it. Incremental graph updates — recomputing metrics for the affected neighbourhood rather than the whole graph — would remove the principal scalability constraint and are a prerequisite for the streaming detection path that would allow a transaction to be scored as it arrives rather than after the fact. On the engineering side, pinning dependency versions, archiving each evaluation's result files with the commit that produced them, and committing the training-only Isolation Forest contamination estimate would make every figure in this report reproducible from the repository alone.

In the medium term, multi-tenant support through institution-namespaced data, an analyst feedback loop in which case resolutions are fed back as labels for model retraining, and configurable rule thresholds exposed through the interface rather than fixed in code would all move the system closer to operational use. Further ahead, temporal graph modelling that treats the transaction network as evolving over time rather than as a single static snapshot, and confidence scoring for ring assembly, would address the two most significant methodological limitations identified in Chapter 7.

The foundational architecture — a graph representation of the transaction history, a layered detection pipeline with independently runnable stages, an ORM-mediated persistence layer, and a separation between detection and investigation — is well suited to support all of these enhancements without requiring a re-architecture. Future work will focus on progressively addressing the limitations identified in Chapter 7, beginning with evaluation on real labelled data and camouflage-resistant graph learning.

Bibliography

[1] L. Page, S. Brin, R. Motwani, and T. Winograd, “The PageRank Citation Ranking: Bringing Order to the Web,” Stanford InfoLab, Technical Report 1999-66, 1999.

[2] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, “Fast unfolding of communities in large networks,” *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2008, no. 10, P10008, 2008. doi: 10.1088/1742-5468/2008/10/P10008

[3] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. doi: 10.1023/A:1010933404324

[4] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 785–794. doi: 10.1145/2939672.2939785

[5] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation Forest,” in *Proc. 8th IEEE Int. Conf. on Data Mining (ICDM)*, 2008, pp. 413–422. doi: 10.1109/ICDM.2008.17

[6] T. N. Kipf and M. Welling, “Semi-Supervised Classification with Graph Convolutional Networks,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2017. [Online]. Available: `https://arxiv.org/abs/1609.02907`

[7] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive Representation Learning on Large Graphs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017. [Online]. Available: `https://arxiv.org/abs/1706.02216`

[8] M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson, and C. E. Leiserson, “Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics,” in *Proc. KDD Workshop on Anomaly Detection in Finance*, 2019. [Online]. Available: `https://arxiv.org/abs/1908.02591`

[9] Y. Dou, Z. Liu, L. Sun, Y. Deng, H. Peng, and P. S. Yu, “Enhancing Graph Neural Network-based Fraud Detectors against Camouflaged Fraudsters,” in *Proc. 29th*

ACM Int. Conf. on Information and Knowledge Management (CIKM), 2020, pp. 315–324. doi: 10.1145/3340531.3411903

[10] T. Pourhabibi, K.-L. Ong, B. H. Kam, and Y. L. Boo, "Fraud detection: A systematic literature review of graph-based anomaly detection approaches," *Decision Support Systems*, vol. 133, 113303, 2020. doi: 10.1016/j.dss.2020.113303

[11] D. Cheng, Y. Zou, S. Xiang, and C. Jiang, "Graph neural networks for financial fraud detection: a review," *Frontiers of Computer Science*, vol. 19, no. 9, art. 199609, 2025. doi: 10.1007/s11704-024-40474-y

[12] R. J. Bolton and D. J. Hand, "Statistical Fraud Detection: A Review," *Statistical Science*, vol. 17, no. 3, pp. 235–255, 2002. doi: 10.1214/ss/1042727940

[13] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring Network Structure, Dynamics, and Function using NetworkX," in *Proc. 7th Python in Science Conf. (SciPy)*, 2008, pp. 11–15.

[14] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[15] M. Fey and J. E. Lenssen, "Fast Graph Representation Learning with PyTorch Geometric," in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019. [Online]. Available: `https://arxiv.org/abs/1903.02428`

[16] S. Ramírez, "FastAPI," 2019. [Online]. Available: `https://fastapi.tiangolo.com/` (accessed Oct. 18, 2026).

[17] M. Bayer, "SQLAlchemy — The Python SQL Toolkit and Object Relational Mapper," 2006. [Online]. Available: `https://www.sqlalchemy.org/` (accessed Oct. 18, 2026).

[18] Financial Intelligence Unit — India, "Obligations of Reporting Entities under the Prevention of Money Laundering Act, 2002 and the Rules thereunder," Department of Revenue, Ministry of Finance, Government of India. [Online]. Available: `https://fiuindia.gov.in/` (accessed Oct. 21, 2026).